

448_Project_Stroke_Data

Anurag Nepal

2025-03-16

```
stroke_data = read.csv("healthcare-dataset-stroke-data.csv")
```

```
glimpse(stroke_data)
```

```
## Rows: 5,110
## Columns: 12
## $ id          <int> 9046, 51676, 31112, 60182, 1665, 56669, 53882, 10434~
## $ gender      <chr> "Male", "Female", "Male", "Female", "Female", "Male"~
## $ age         <dbl> 67, 61, 80, 49, 79, 81, 74, 69, 59, 78, 81, 61, 54, ~
## $ hypertension <int> 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1~
## $ heart_disease <int> 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0~
## $ ever_married <chr> "Yes", "Yes", "Yes", "Yes", "Yes", "Yes", "Yes", "No~
## $ work_type    <chr> "Private", "Self-employed", "Private", "Private", "S~
## $ Residence_type <chr> "Urban", "Rural", "Rural", "Urban", "Rural", "Urban"~
## $ avg_glucose_level <dbl> 228.69, 202.21, 105.92, 171.23, 174.12, 186.21, 70.0~
## $ bmi          <chr> "36.6", "N/A", "32.5", "34.4", "24", "29", "27.4", "~
## $ smoking_status <chr> "formerly smoked", "never smoked", "never smoked", "~
## $ stroke       <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
```

```
summary(stroke_data)
```

```
##           id           gender           age           hypertension
## Min.      : 67   Length:5110   Min.      : 0.08   Min.      :0.00000
## 1st Qu.:17741   Class :character   1st Qu.:25.00   1st Qu.:0.00000
## Median :36932   Mode  :character   Median :45.00   Median :0.00000
## Mean      :36518                                Mean      :43.23   Mean      :0.09746
## 3rd Qu.:54682                                3rd Qu.:61.00   3rd Qu.:0.00000
## Max.      :72940                                Max.      :82.00   Max.      :1.00000
## heart_disease   ever_married           work_type           Residence_type
## Min.      :0.00000   Length:5110           Length:5110           Length:5110
## 1st Qu.:0.00000   Class :character       Class :character       Class :character
## Median :0.00000   Mode  :character       Mode  :character       Mode  :character
## Mean      :0.05401                                Mean      :0.05401
## 3rd Qu.:0.00000                                3rd Qu.:0.00000
## Max.      :1.00000                                Max.      :1.00000
## avg_glucose_level   bmi           smoking_status           stroke
## Min.      : 55.12   Length:5110           Length:5110           Min.      :0.00000
## 1st Qu.: 77.25     Class :character       Class :character       1st Qu.:0.00000
## Median : 91.89     Mode  :character       Mode  :character       Median :0.00000
## Mean      :106.15                                Mean      :0.04873
## 3rd Qu.:114.09                                3rd Qu.:0.00000
## Max.      :271.74                                Max.      :1.00000
```

BMI should be integer but is there as character.

```
stroke_data$bmi=as.double(stroke_data$bmi)
```

```
## Warning: NAs introduced by coercion
```

```
summary(stroke_data)
```

```
##          id          gender          age          hypertension
## Min.      : 67   Length:5110   Min.      : 0.08   Min.      :0.00000
## 1st Qu.:17741   Class :character 1st Qu.:25.00   1st Qu.:0.00000
## Median :36932   Mode  :character  Median :45.00   Median :0.00000
## Mean    :36518                Mean  :43.23   Mean    :0.09746
## 3rd Qu.:54682                3rd Qu.:61.00   3rd Qu.:0.00000
## Max.     :72940                Max.     :82.00   Max.     :1.00000
##
## heart_disease  ever_married      work_type      Residence_type
## Min.          :0.00000   Length:5110   Length:5110   Length:5110
## 1st Qu.:0.00000   Class :character  Class :character  Class :character
## Median :0.00000   Mode  :character  Mode  :character  Mode  :character
## Mean      :0.05401
## 3rd Qu.:0.00000
## Max.      :1.00000
##
## avg_glucose_level  bmi      smoking_status      stroke
## Min.      : 55.12   Min.      :10.30   Length:5110   Min.      :0.00000
## 1st Qu.: 77.25   1st Qu.:23.50   Class :character 1st Qu.:0.00000
## Median : 91.89   Median :28.10   Mode  :character Median :0.00000
## Mean     :106.15   Mean      :28.89                Mean     :0.04873
## 3rd Qu.:114.09   3rd Qu.:33.10                3rd Qu.:0.00000
## Max.     :271.74   Max.      :97.60                Max.     :1.00000
##
##                      NA's      :201
```

```
stroke_counts = table(stroke_data$stroke)
stroke_counts_df = as.data.frame(stroke_counts)
stroke_counts_df$percentage = stroke_counts_df$Freq / sum(stroke_counts_df$Freq) * 100
colnames(stroke_counts_df) = c("stroke", "count", "percentage")
print(stroke_counts_df)
```

```
##   stroke count percentage
## 1      0  4861  95.127202
## 2      1   249   4.872798
```

Data imbalance so may have to augment the data.

```
na_counts = colSums(is.na(stroke_data))
na_counts[na_counts > 0]
```

```
## bmi
## 201
```

```
stroke_data = stroke_data[stroke_data$gender != "Other", ]
```

```
categorize_age = function(age) {
  if (age >= 0 & age <= 1) {
    return('Infant')
  } else if (age > 1 & age <= 12) {
    return('Children')
  } else if (age > 12 & age < 20) {
    return('Teenager')
  } else if (age >= 20 & age <= 39) {
    return('Adult')
  } else if (age > 39 & age <= 59) {
    return('Middle Aged')
  } else {
    return('Senior')
  }
}
```

```
stroke_data$age_group = sapply(stroke_data$age, categorize_age)
unique(stroke_data$age_group)
```

```
## [1] "Senior"      "Middle Aged" "Adult"      "Children"    "Teenager"
## [6] "Infant"
```

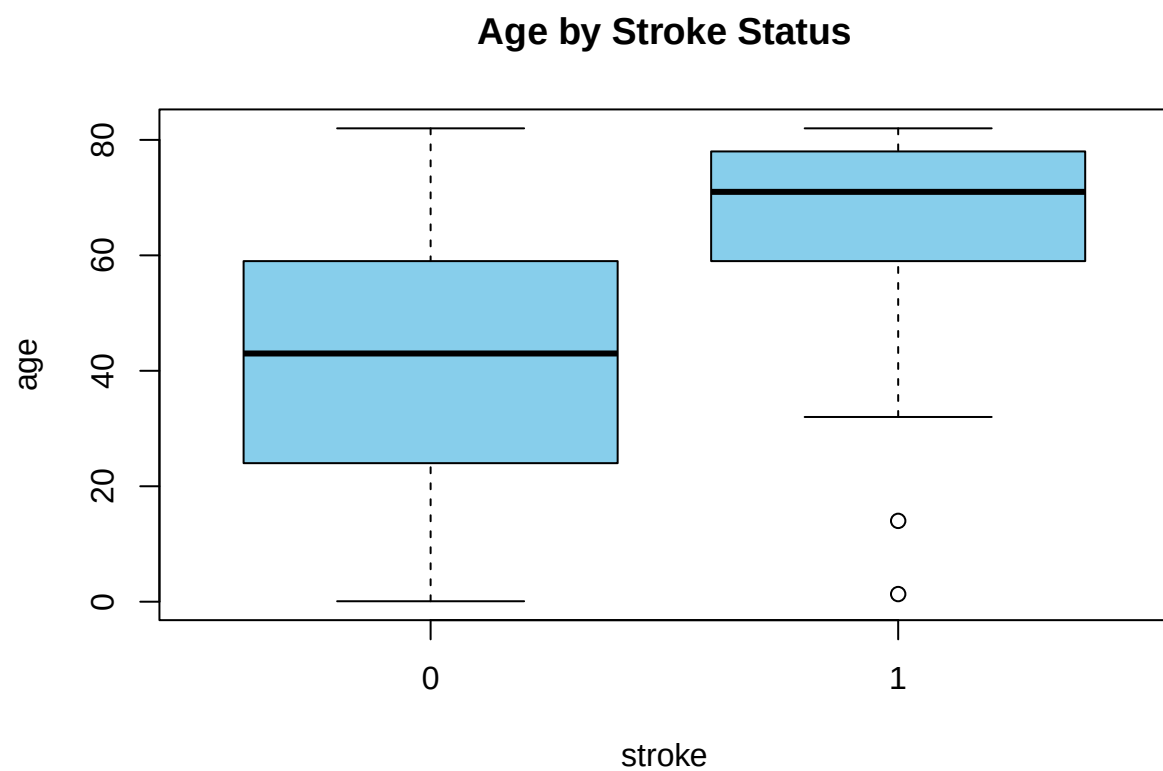
```
avg_bmi_by_group = aggregate(bmi ~ age_group + gender, data = stroke_data, FUN = mean)
colnames(avg_bmi_by_group) = c("age_group", "gender", "avg_bmi")
avg_bmi_by_group
```

```
##      age_group gender  avg_bmi
## 1      Adult Female 29.68774
## 2    Children Female 19.23281
## 3      Infant Female 18.21250
## 4 Middle Aged Female 31.40455
## 5      Senior Female 29.90319
## 6    Teenager Female 26.27750
## 7      Adult   Male 30.01599
## 8    Children   Male 19.67847
## 9      Infant   Male 18.43871
## 10 Middle Aged   Male 31.90985
## 11      Senior   Male 30.06266
## 12    Teenager   Male 25.67471
```

```
stroke_data = merge(stroke_data, avg_bmi_by_group, by = c("age_group", "gender"), all.x = TRUE)
stroke_data$bmi = ifelse(is.na(stroke_data$bmi), stroke_data$avg_bmi, stroke_data$bmi)
stroke_data$avg_bmi = NULL
```

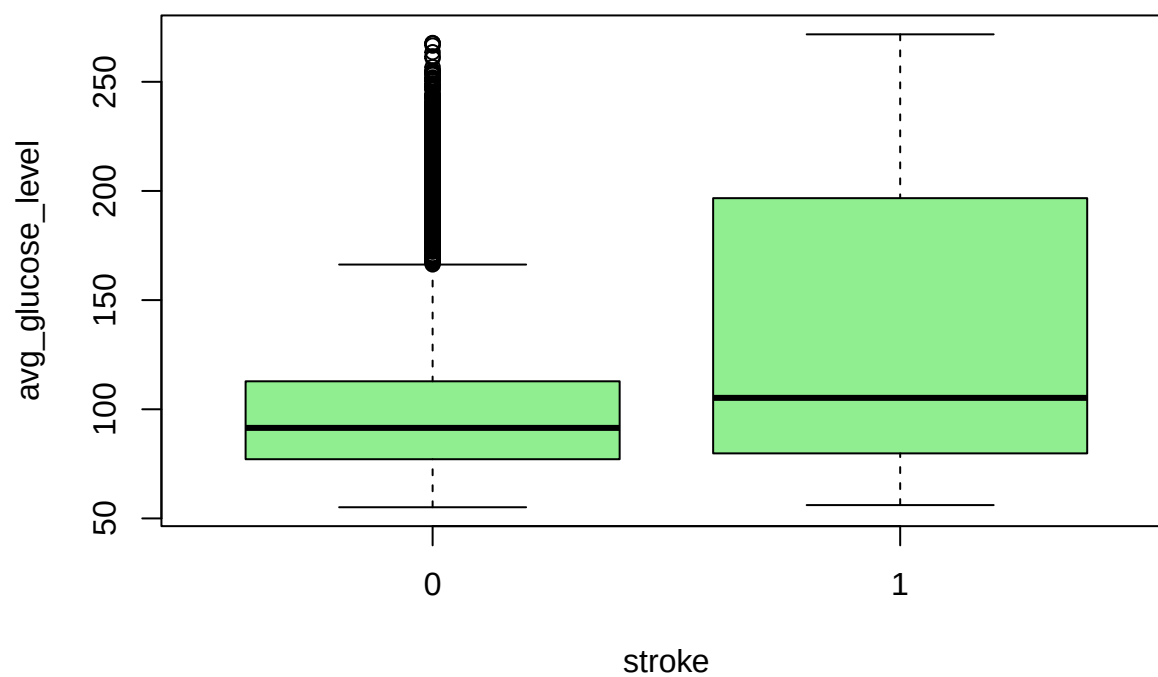
Used the average bmi for the age group and gender to fill in the bmi NA values.

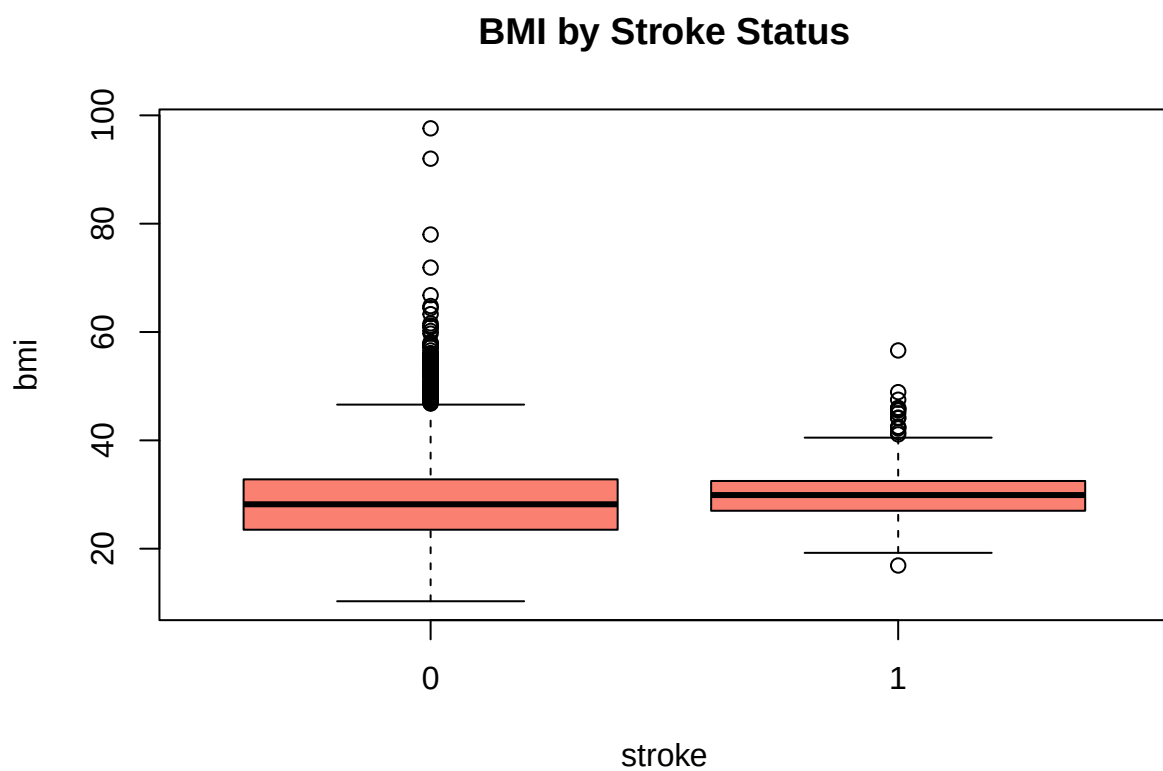
```
boxplot(age ~ stroke, data = stroke_data, main = "Age by Stroke Status", col = "skyblue")
```



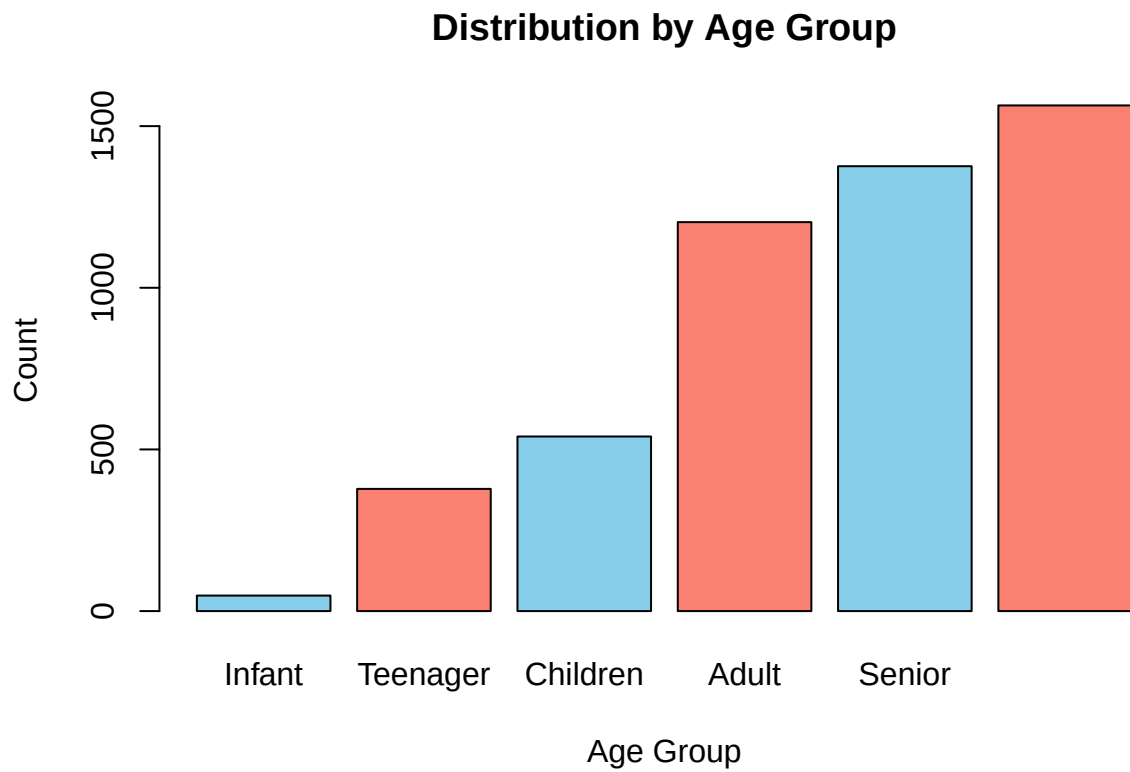
```
boxplot(avg_glucose_level ~ stroke, data = stroke_data, main = "Glucose Level by Stroke Status", col =
```

Glucose Level by Stroke Status



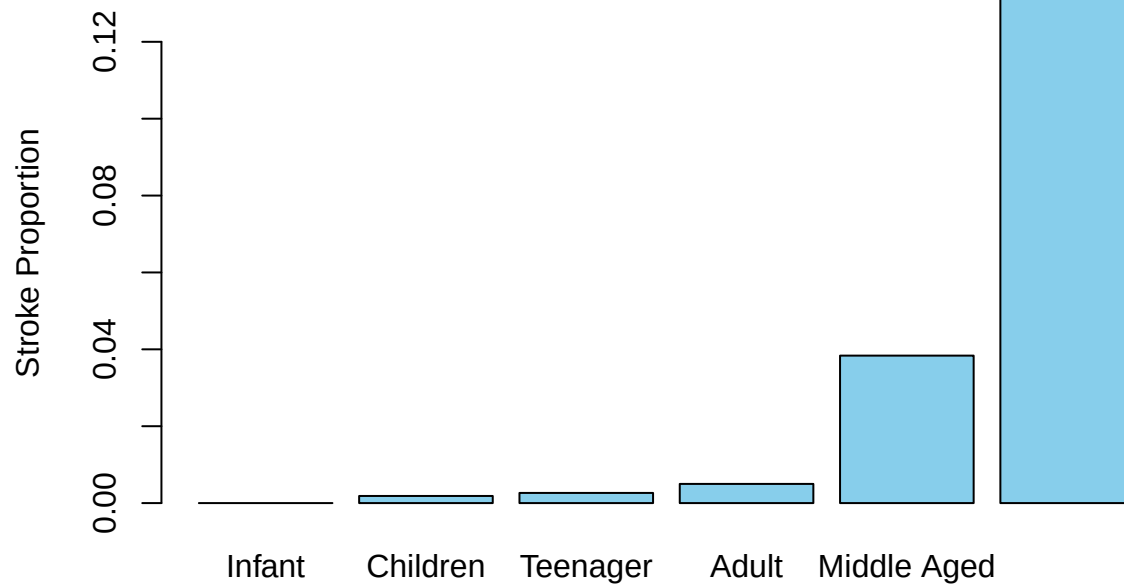


```
barplot(sort(table(stroke_data$age_group)), beside = TRUE, main = "Distribution by Age Group",  
        xlab = "Age Group", ylab = "Count", col = c("skyblue", "salmon"),  
        legend = rownames(stroke_data$age_group))
```

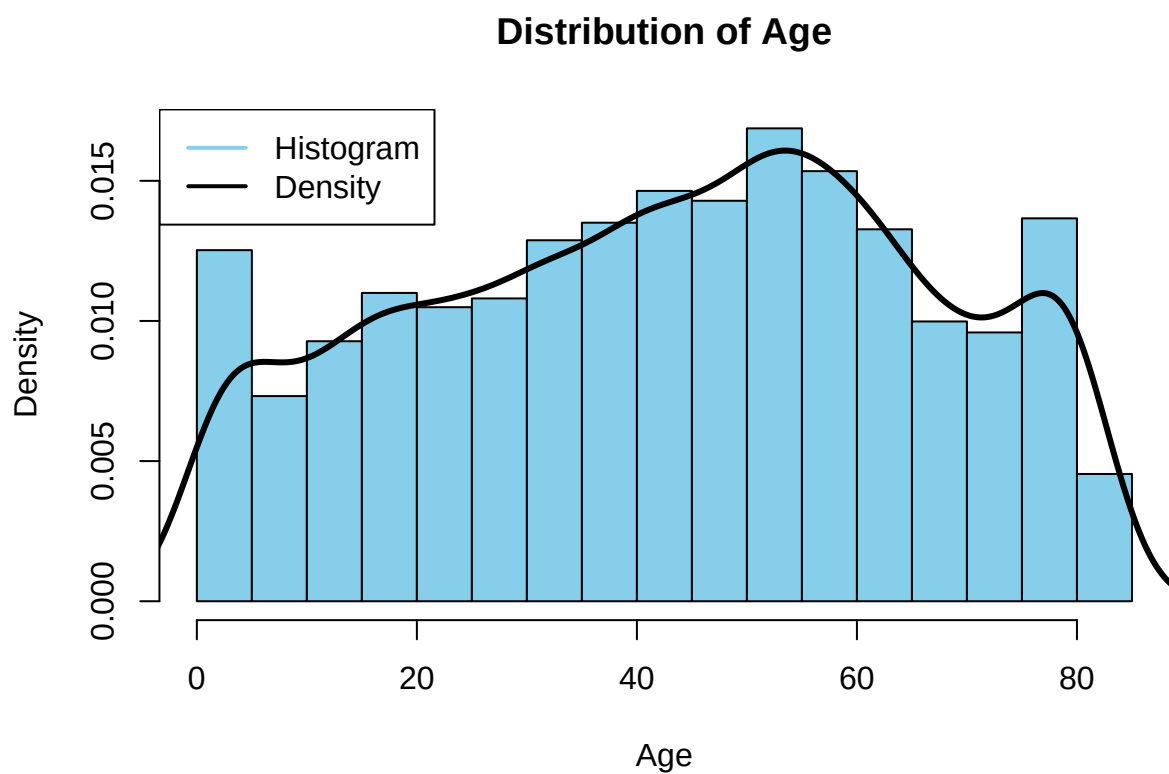


```
age_group_stroke = prop.table(table(stroke_data$age_group, stroke_data$stroke), margin = 1)
barplot(sort(age_group_stroke[, 2]), col = "skyblue", main = "Stroke Rate by Age Group", ylab = "Stroke
```

Stroke Rate by Age Group

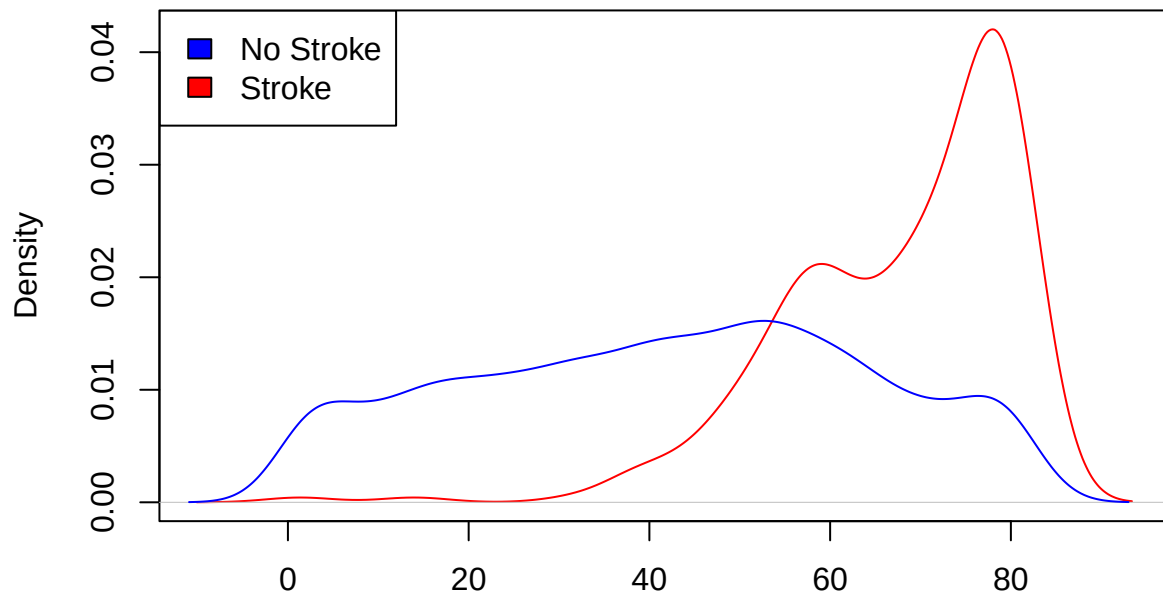


```
hist(stroke_data$Age, main = "Distribution of Age", xlab = "Age", col = "skyblue", breaks = 20, prob = '
lines(density(stroke_data$Age, na.rm = TRUE), col = "black", lwd = 3)
legend("topleft", legend = c("Histogram", "Density"), col = c("skyblue", "black"), lwd = 2)
```



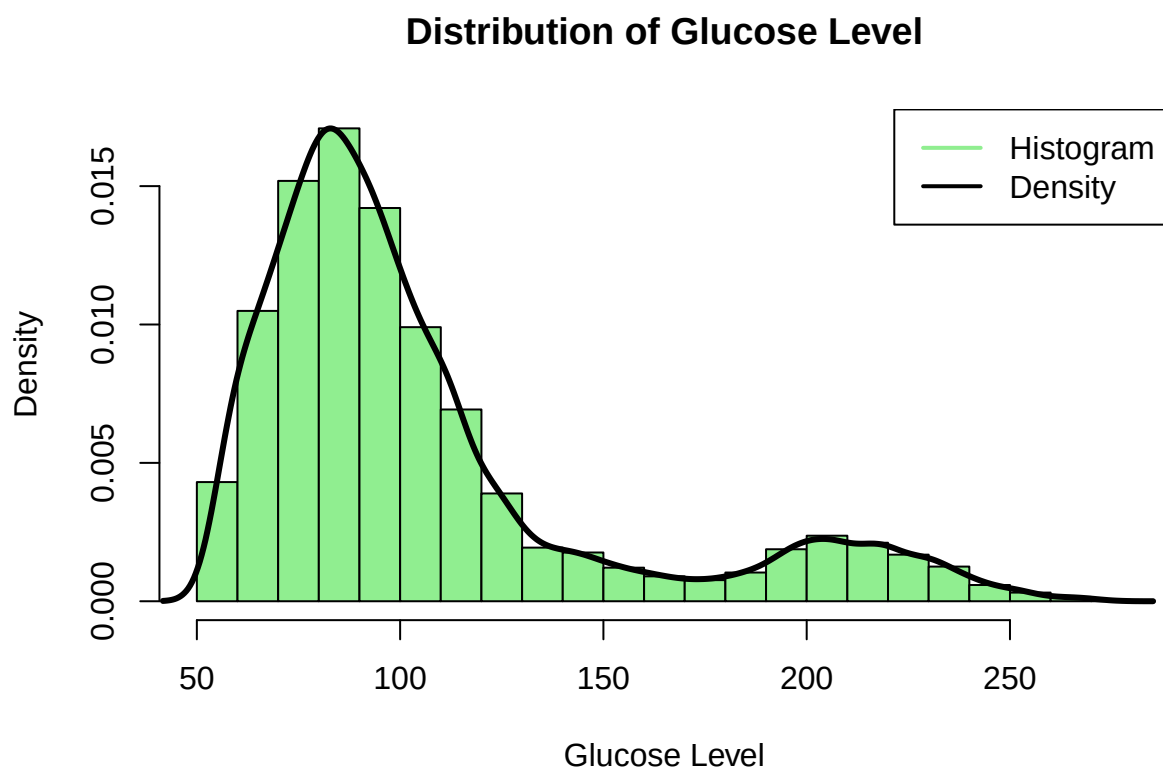
```
plot(density(stroke_data$age[stroke_data$stroke == 1]), col = "red", main = "Age Distribution by Stroke",  
lines(density(stroke_data$age[stroke_data$stroke == 0]), col = "blue")  
legend("topleft", legend = c("No Stroke", "Stroke"), fill = c("blue", "red"))
```

Age Distribution by Stroke Status



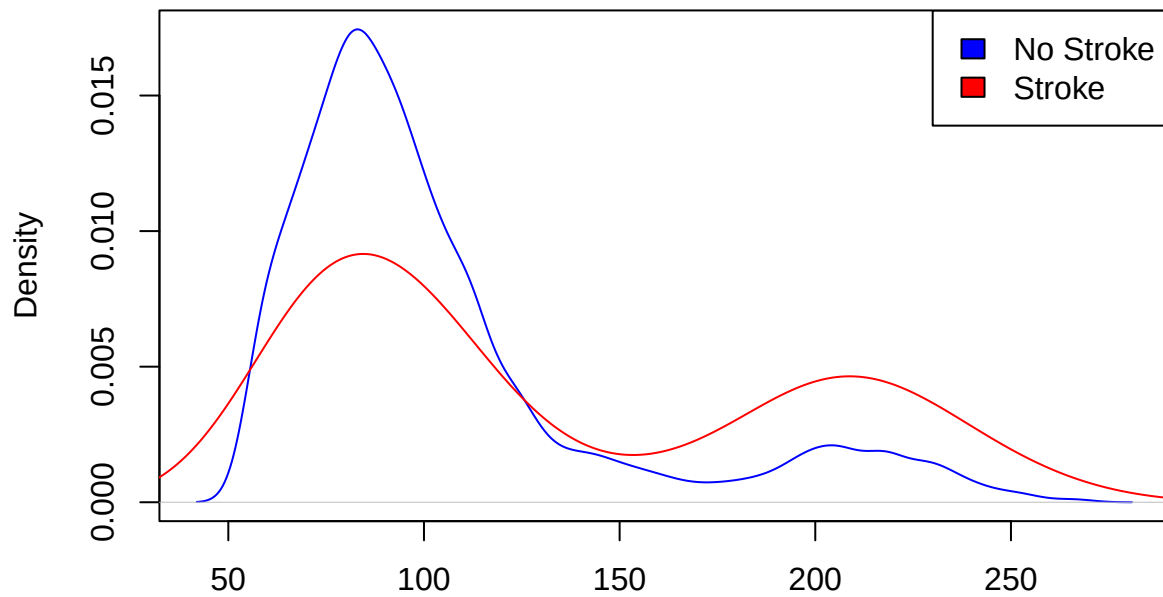
N = 249 Bandwidth = 3.8

```
hist(stroke_data$avg_glucose_level, main = "Distribution of Glucose Level", xlab = "Glucose Level", col = "lightgreen", lwd = 3)
lines(density(stroke_data$avg_glucose_level), col = "black", lwd = 3)
legend("topright", legend = c("Histogram", "Density"), col = c("lightgreen", "black"), lwd = 2)
```

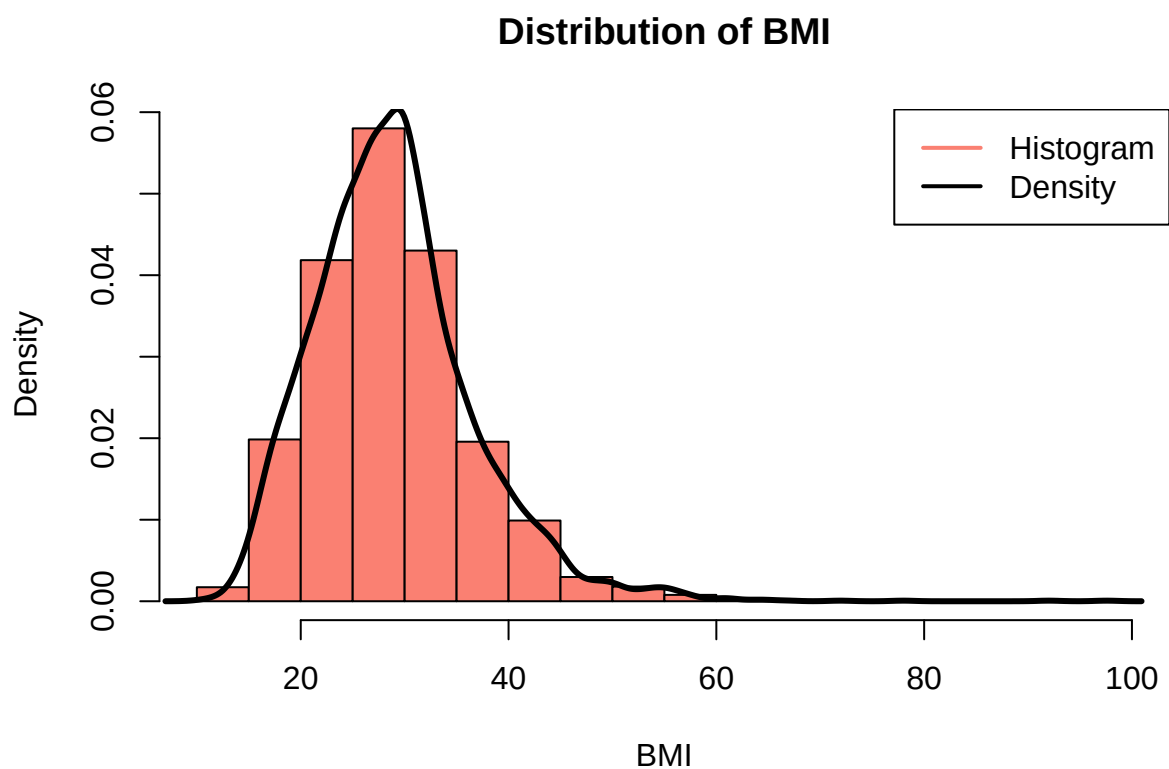


```
plot(density(stroke_data$avg_glucose_level[stroke_data$stroke == 0]), col = "blue", main = "Glucose Level Distribution")
lines(density(stroke_data$avg_glucose_level[stroke_data$stroke == 1]), col = "red")
legend("topright", legend = c("No Stroke", "Stroke"), fill = c("blue", "red"))
```

Glucose Level Distribution by Stroke Status

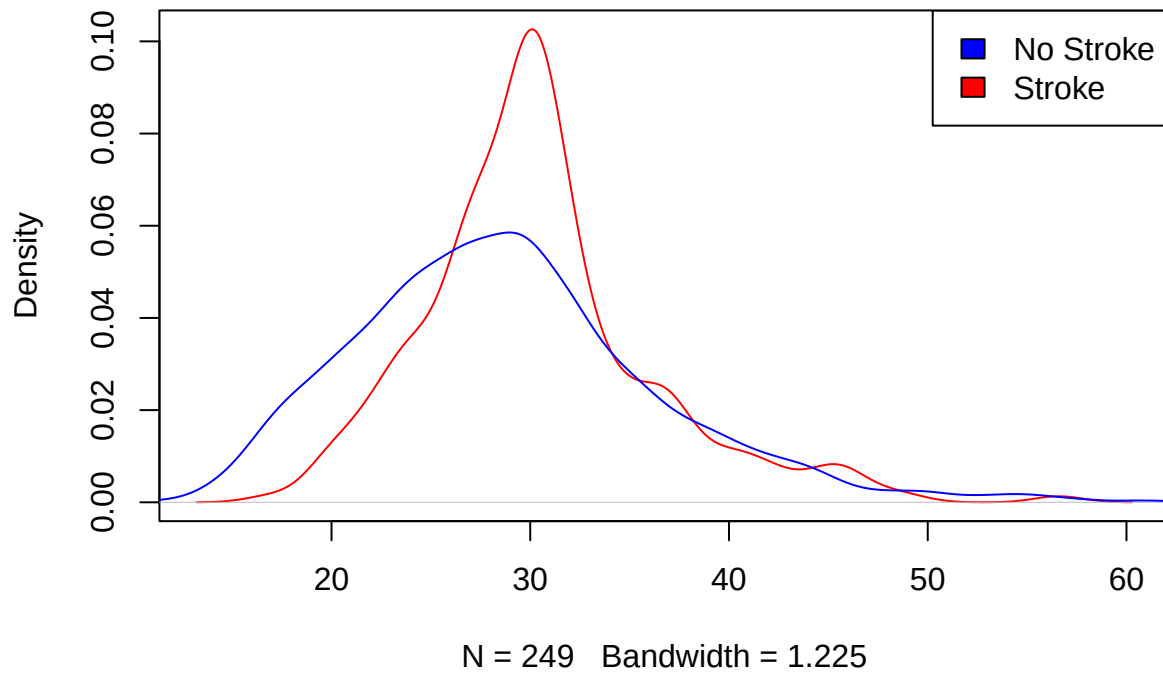


```
hist(stroke_data$bmi, main = "Distribution of BMI", xlab = "BMI", col = "salmon", breaks = 20,prob=TRUE)
lines(density(stroke_data$bmi), col = "black", main = "BMI Distribution",lwd=3)
legend("topright", legend = c("Histogram", "Density"), col = c("salmon", "black"), lwd = 2)
```



```
plot(density(stroke_data$bmi[stroke_data$stroke == 1]), col = "red", main = "BMI Distribution by Stroke",  
lines(density(stroke_data$bmi[stroke_data$stroke == 0]), col = "blue")  
legend("topright", legend = c("No Stroke", "Stroke"), fill = c("blue", "red"))
```

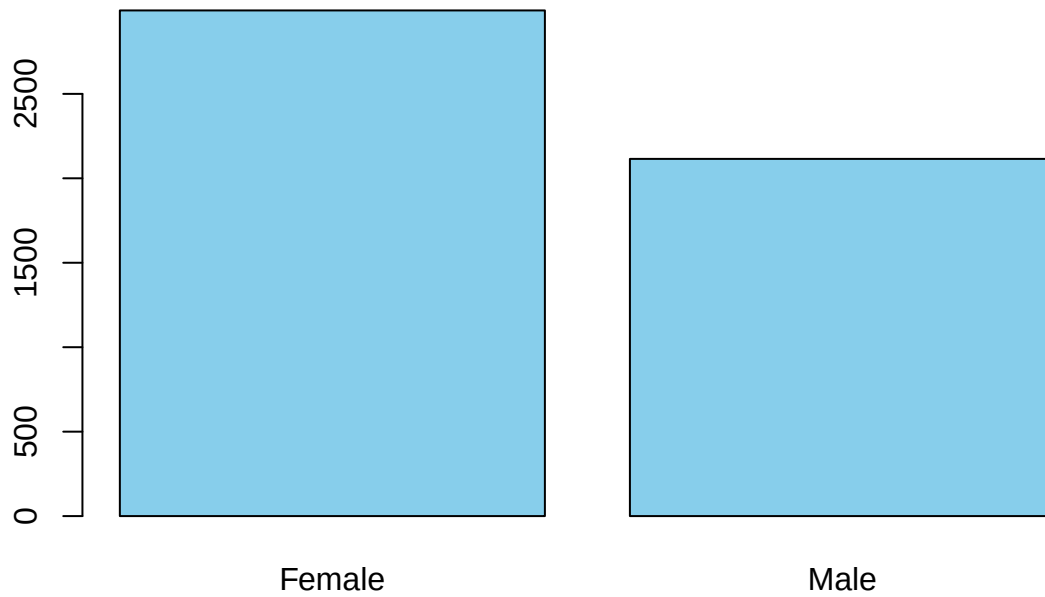
BMI Distribution by Stroke Status



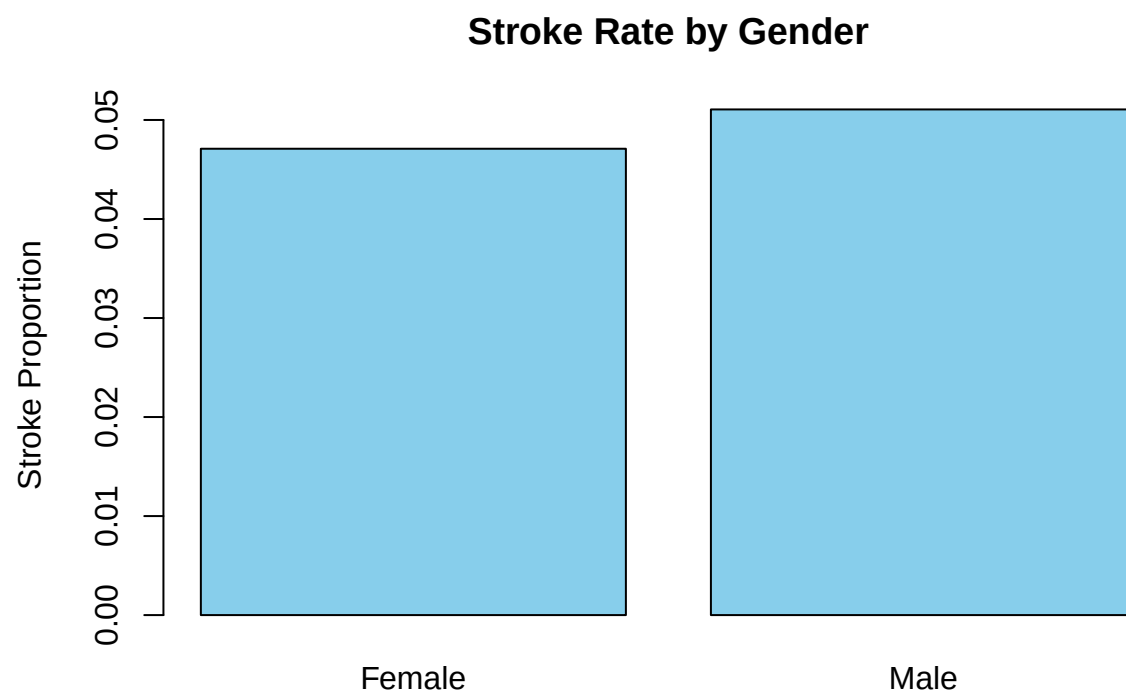
Stroke likelihood increases with lower glucose concentration.

```
barplot(table(stroke_data$gender), main = "Gender Distribution", col = "skyblue")
```

Gender Distribution

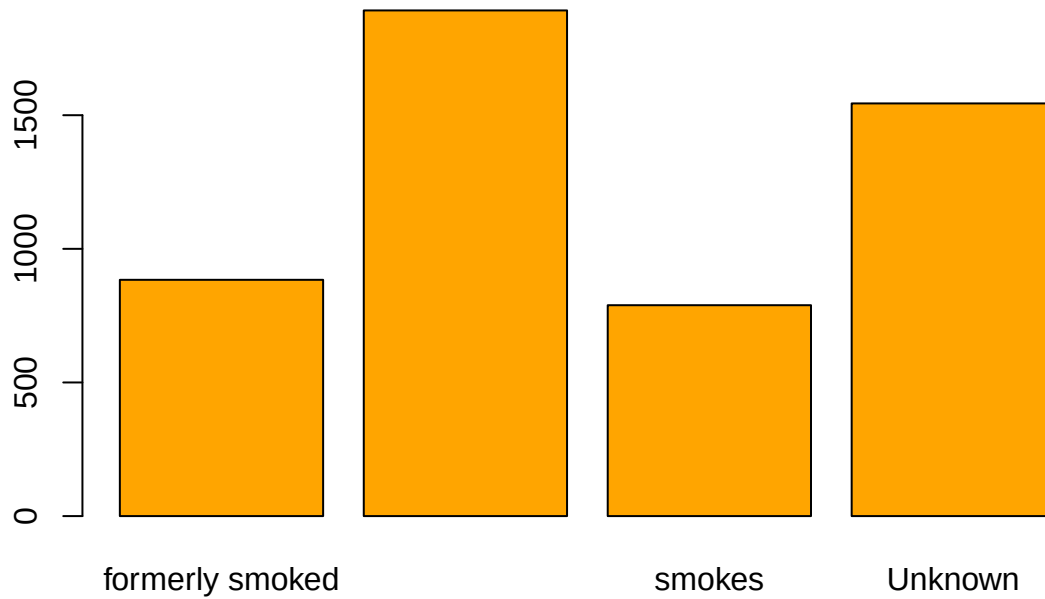


```
gender_stroke = prop.table(table(stroke_data$gender, stroke_data$stroke), margin = 1)
barplot(gender_stroke[, 2], col = "skyblue", main = "Stroke Rate by Gender", ylab = "Stroke Proportion")
```

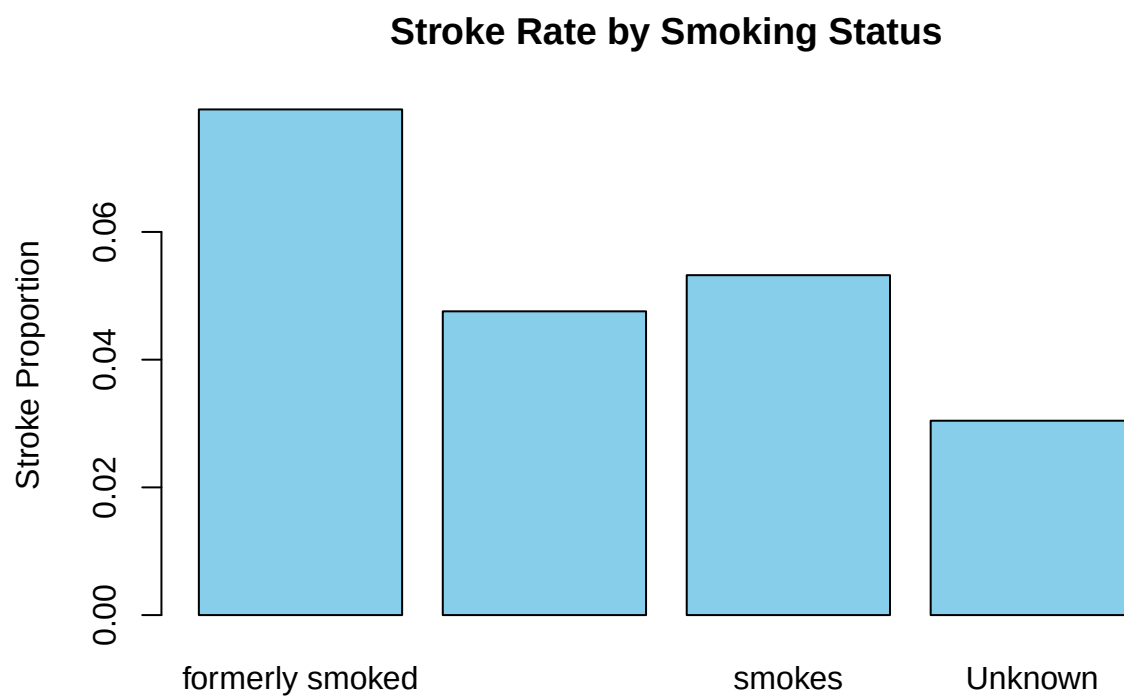


```
barplot(table(stroke_data$smoking_status), main = "Smoking Status Distribution", col = "orange")
```

Smoking Status Distribution



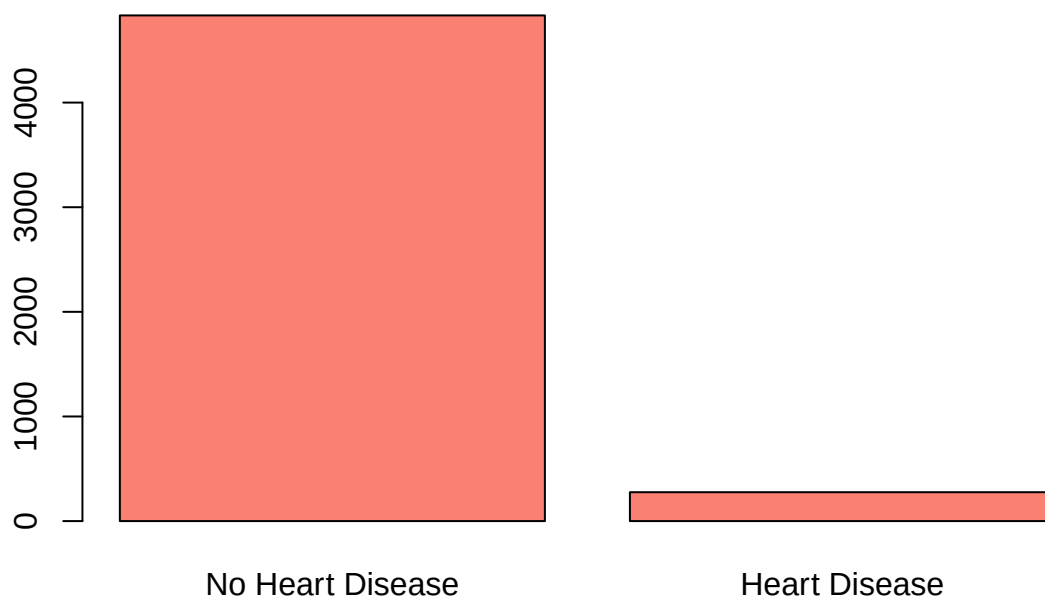
```
smoke_stroke = prop.table(table(stroke_data$smoking_status, stroke_data$stroke), margin = 1)
barplot(smoke_stroke[, 2], col = "skyblue", main = "Stroke Rate by Smoking Status", ylab = "Stroke Prop
```



Former smokers have higher stroke rates.

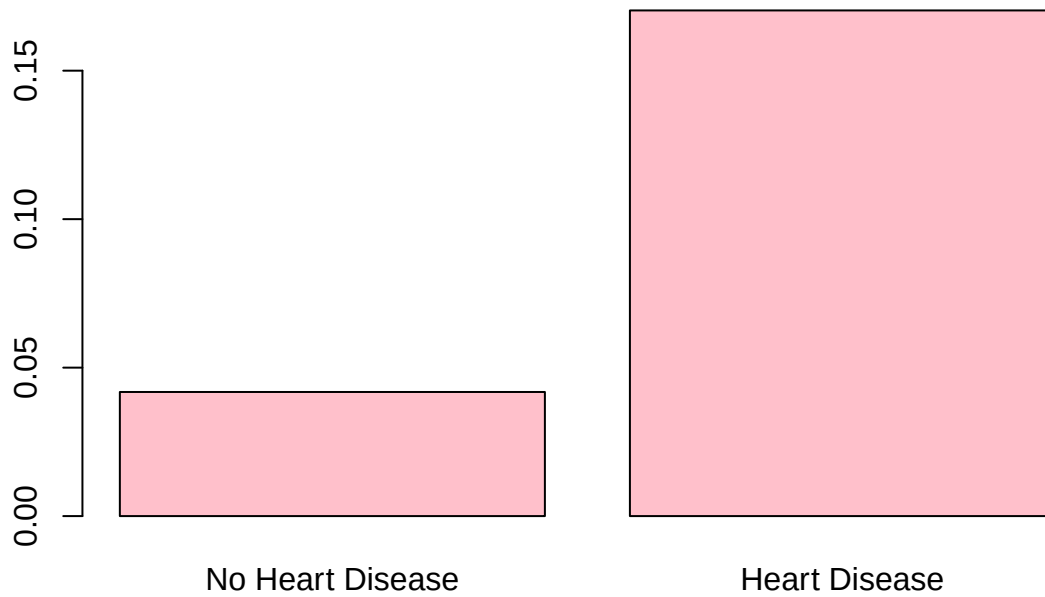
```
barplot(table(stroke_data$heart_disease),names.arg = c("No Heart Disease", "Heart Disease"), main = "Hea
```

Heart Disease Distribution



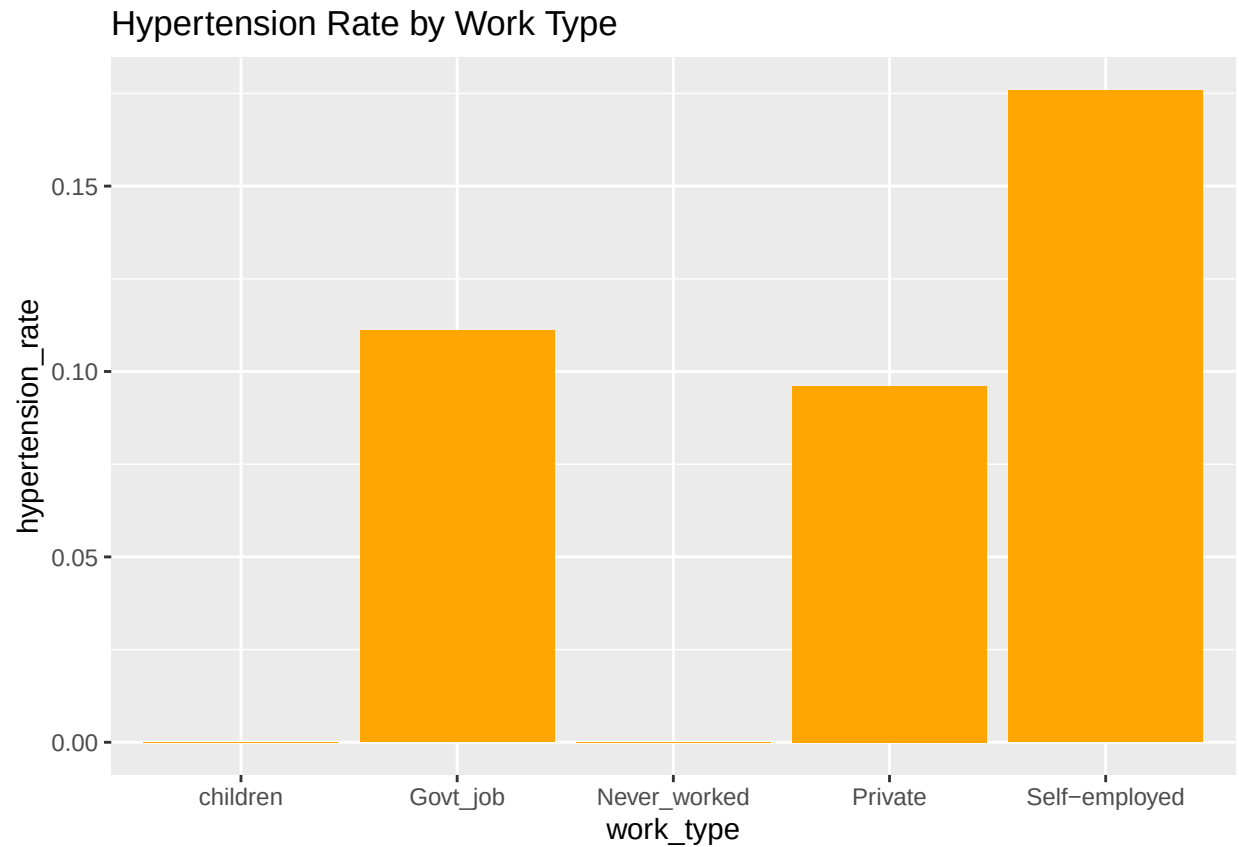
```
hd_stroke = prop.table(table(stroke_data$heart_disease, stroke_data$stroke), margin = 1)
barplot(hd_stroke[, 2], names.arg = c("No Heart Disease", "Heart Disease"), col = "pink", main = "Stroke")
```

Stroke Rate by Heart Disease

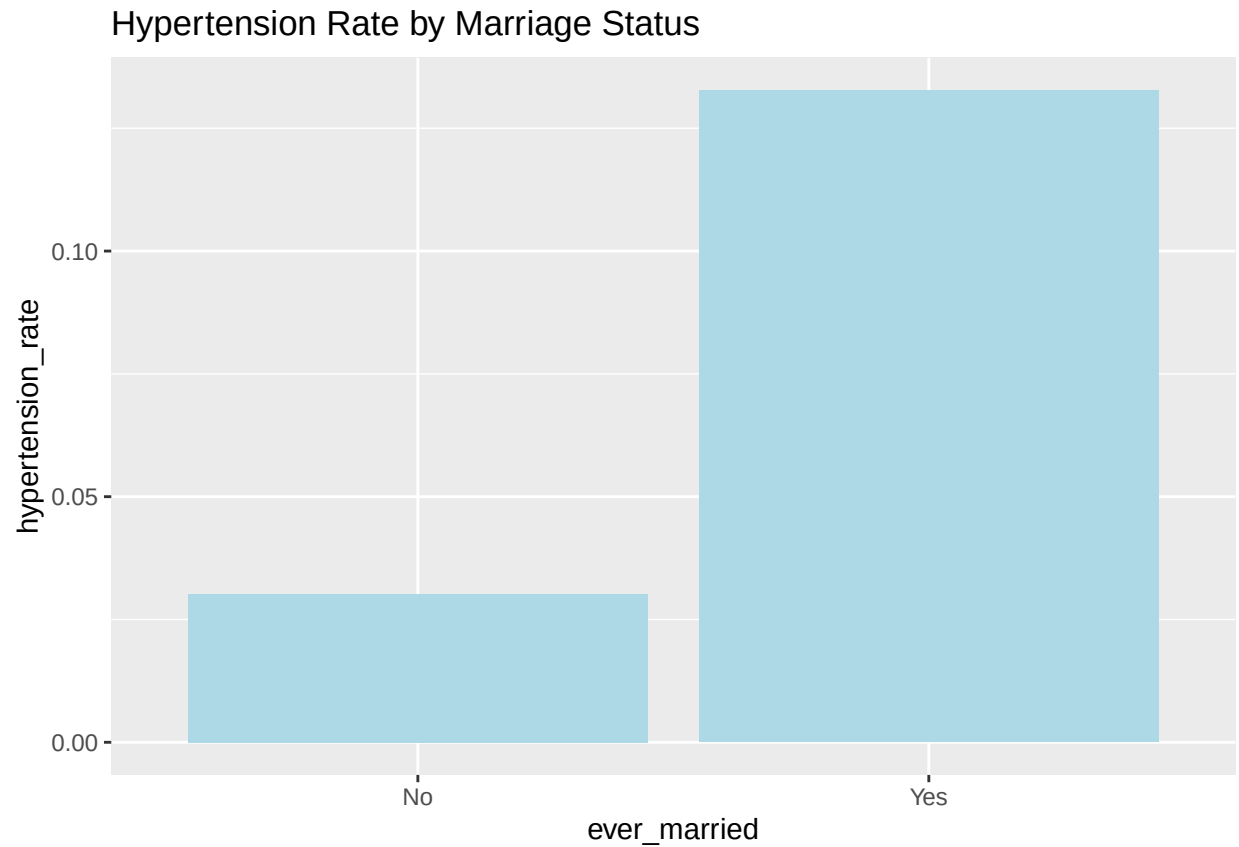


Heart disease increases stroke risk.

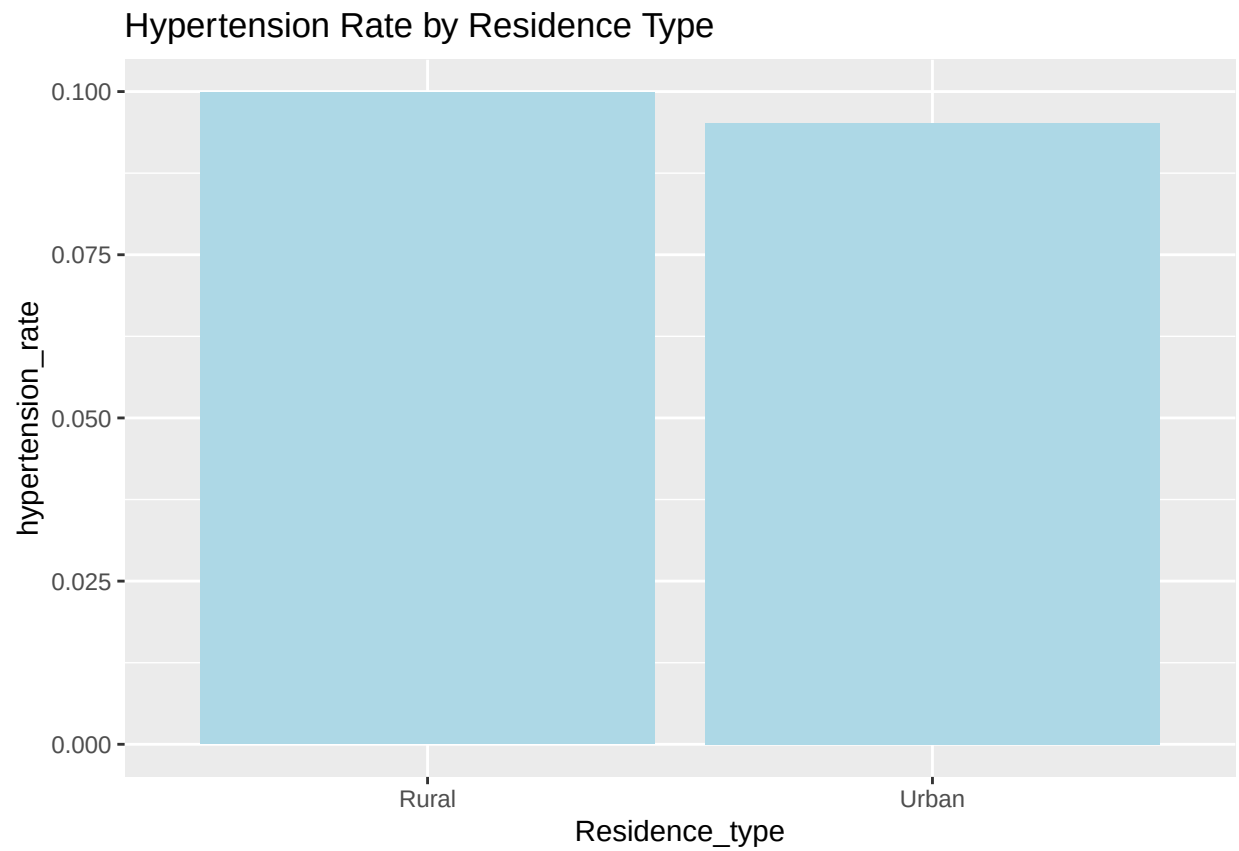
```
stroke_data %>%  
  group_by(work_type) %>%  
  summarise(hypertension_rate = mean(hypertension)) %>%  
  ggplot(aes(work_type, hypertension_rate)) +  
  geom_col(fill = "orange") +  
  labs(title = "Hypertension Rate by Work Type")
```



```
stroke_data %>%  
  group_by(ever_married) %>%  
  summarise(hypertension_rate = mean(hypertension)) %>%  
  ggplot(aes(ever_married, hypertension_rate)) +  
  geom_col(fill = "lightblue") +  
  labs(title = "Hypertension Rate by Marriage Status")
```

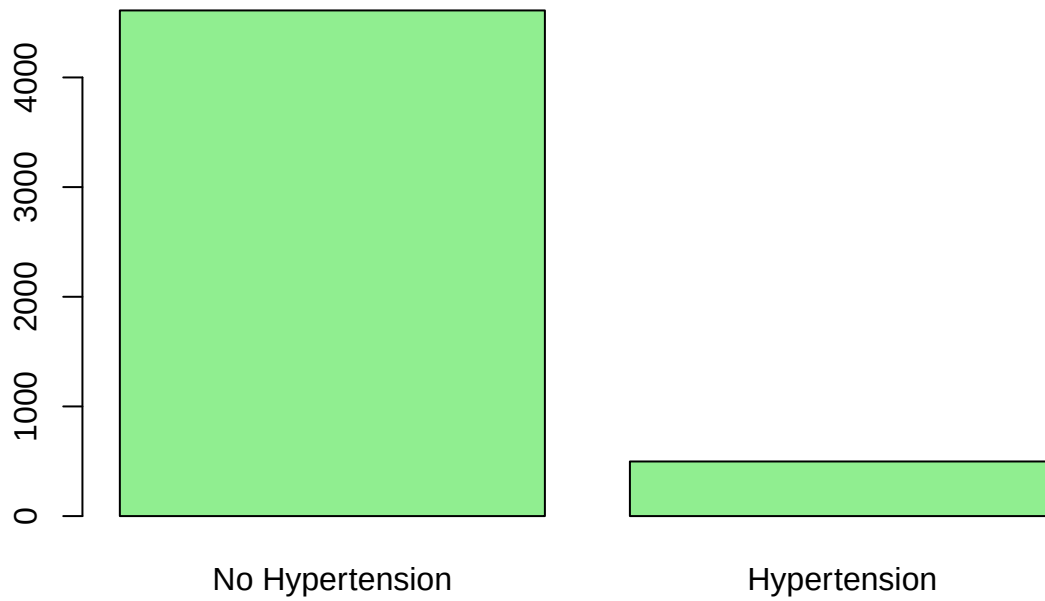


```
stroke_data %>%  
  group_by(Residence_type) %>%  
  summarise(hypertension_rate = mean(hypertension)) %>%  
  ggplot(aes(Residence_type, hypertension_rate)) +  
  geom_col(fill = "lightblue") +  
  labs(title = "Hypertension Rate by Residence Type")
```



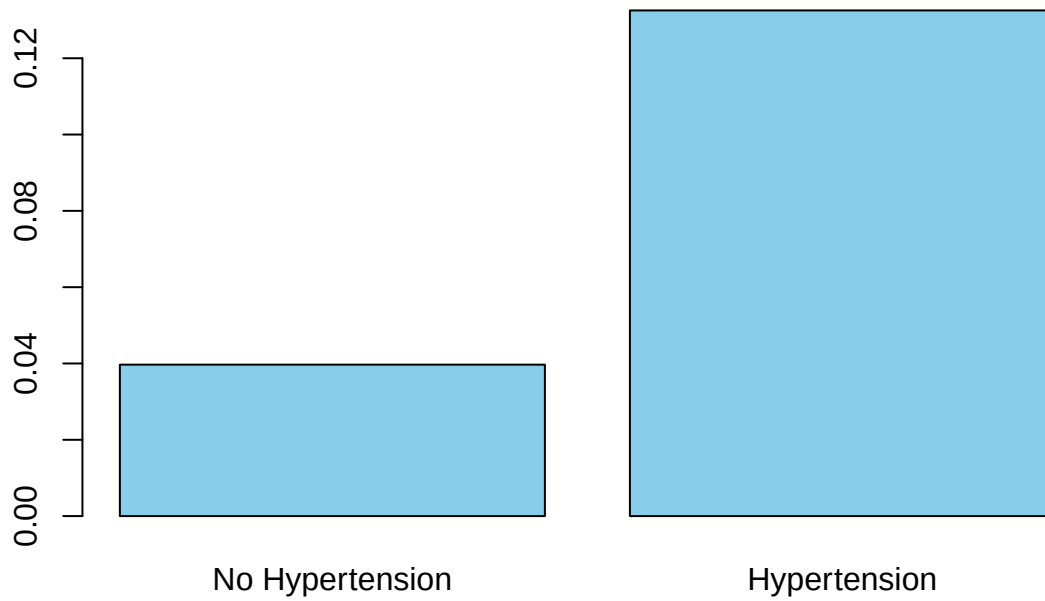
```
barplot(table(stroke_data$hypertension),names.arg = c("No Hypertension", "Hypertension"), main = "Hypertension Rate by Residence Type")
```

Hypertension Distribution



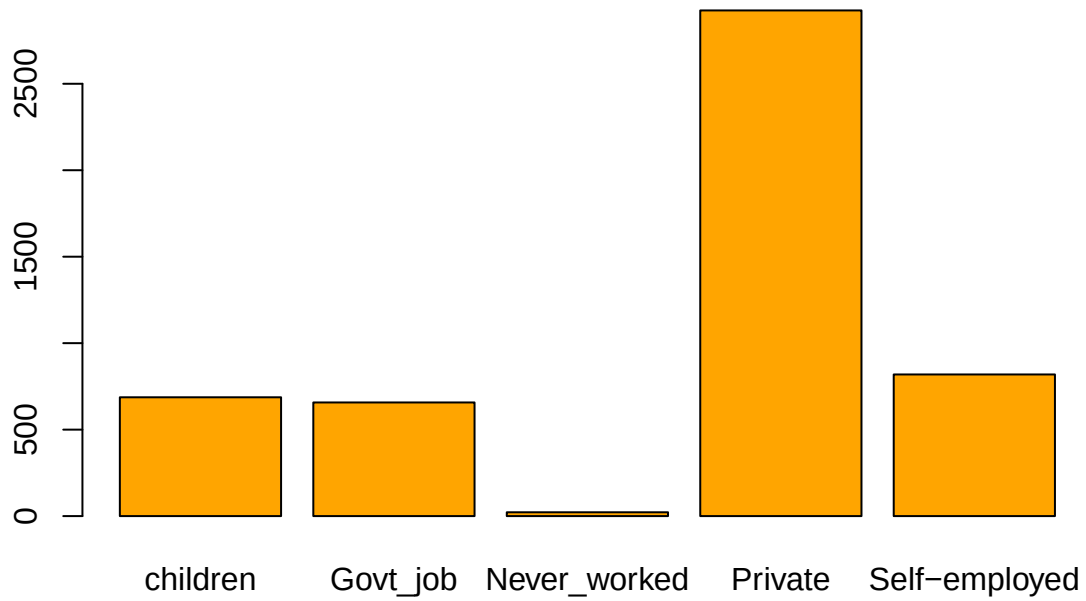
```
hypertension_stroke = prop.table(table(stroke_data$hypertension, stroke_data$stroke), margin = 1)
barplot(hypertension_stroke[, 2], names.arg = c("No Hypertension", "Hypertension"), col = "skyblue", ma
```

Stroke Rate by Hypertension



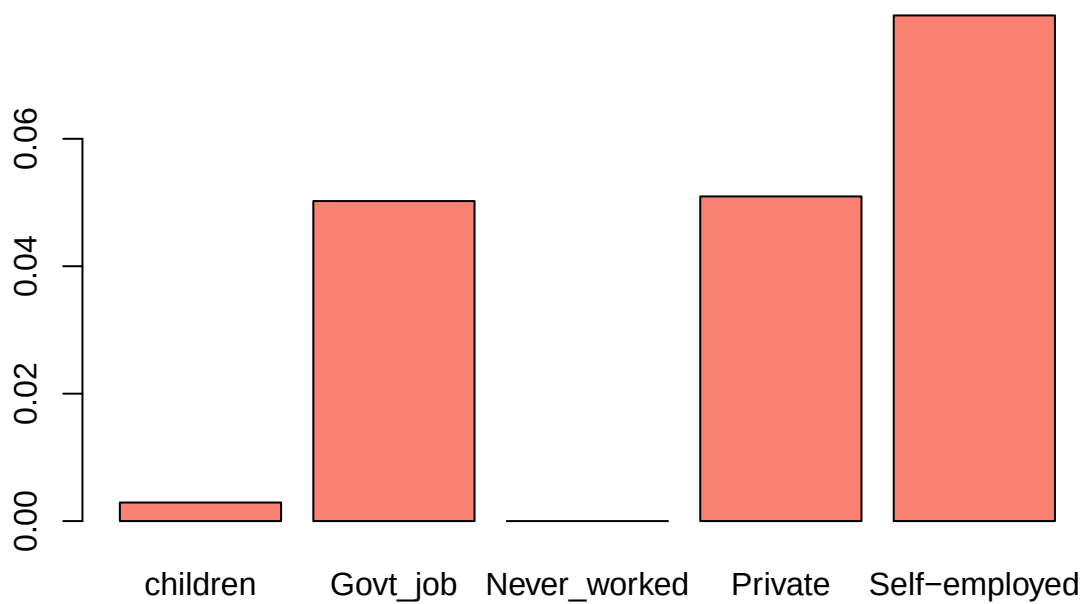
```
barplot(table(stroke_data$work_type), main = "Work Type Distribution", col = "orange")
```

Work Type Distribution



```
res_stroke = prop.table(table(stroke_data$work_type, stroke_data$stroke), margin = 1)
barplot(res_stroke[, 2], col = "salmon", main = "Stroke Rate by Work Type")
```

Stroke Rate by Work Type

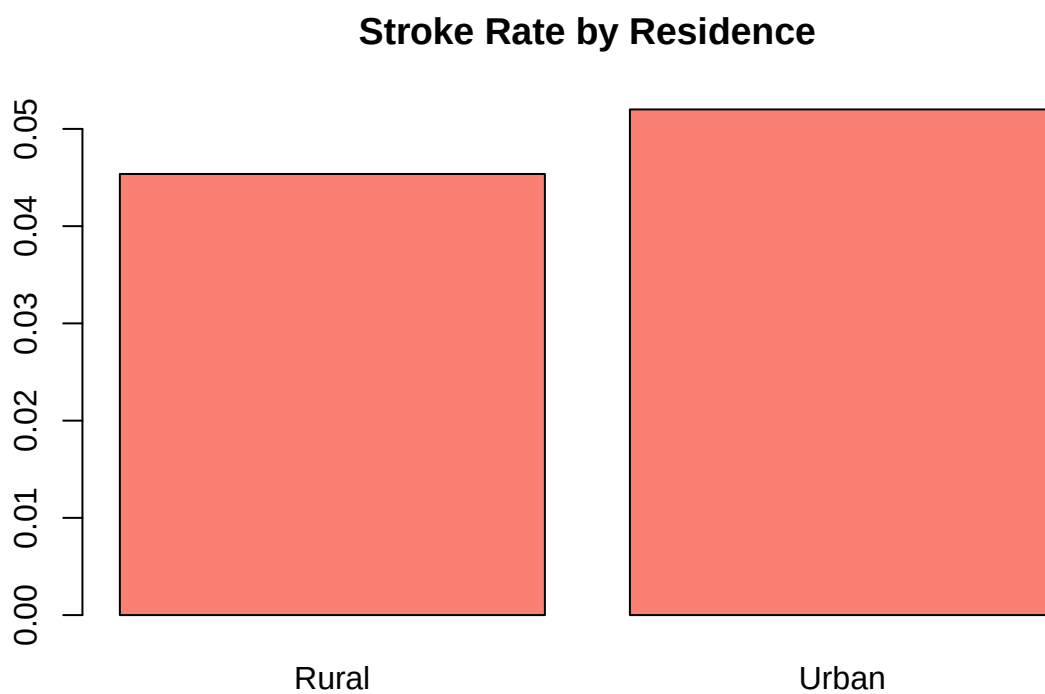


```
barplot(table(stroke_data$Residence_type), main = "Residence Distribution", col = "orange")
```

Residence Distribution

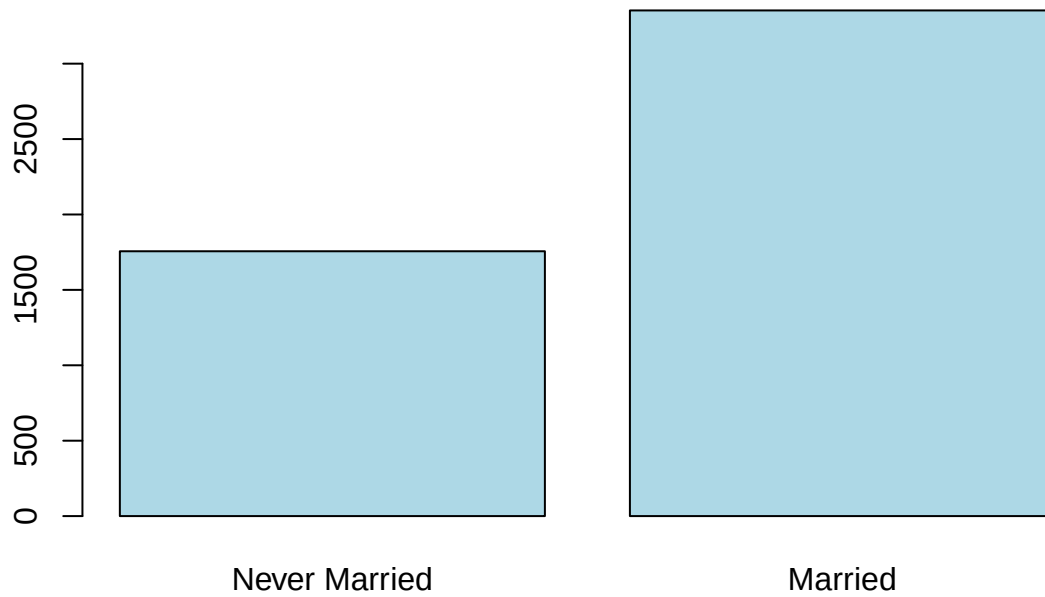


```
res_stroke = prop.table(table(stroke_data$Residence_type, stroke_data$stroke), margin = 1)
barplot(res_stroke[, 2], names.arg = c("Rural", "Urban"), col = "salmon", main = "Stroke Rate by Residence Type")
```



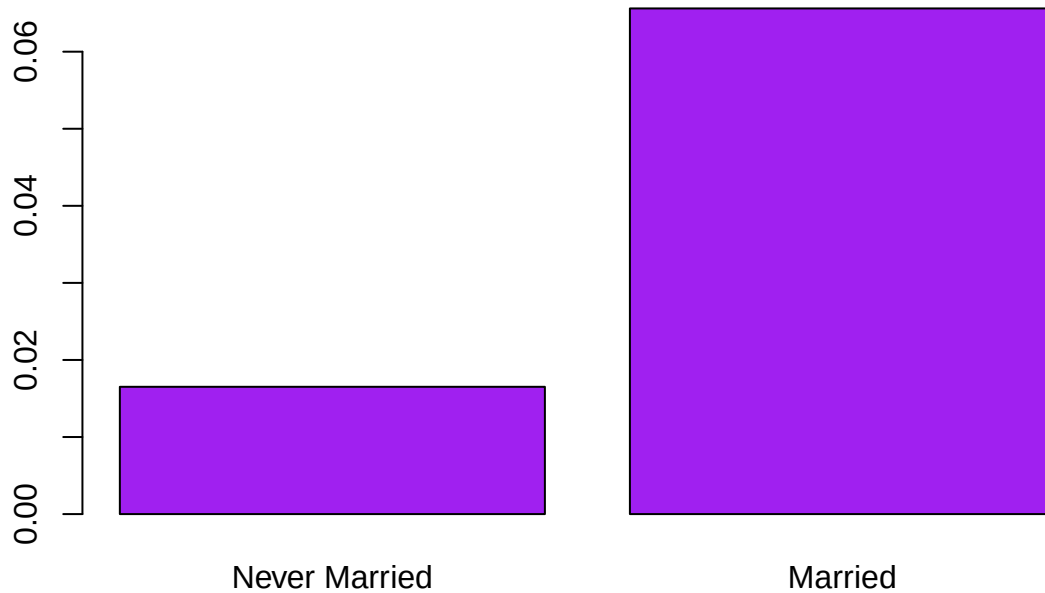
```
barplot(table(stroke_data$ever_married),names.arg = c("Never Married", "Married"), main = " Marital Sta
```

Marital Status Distribution



```
married_stroke = prop.table(table(stroke_data$ever_married, stroke_data$stroke), margin = 1)
barplot(married_stroke[, 2], names.arg = c("Never Married", "Married"), col = "purple", main = "Stroke I
```

Stroke Rate by Marital Status



```
numerical_vars = stroke_data[, c("age", "avg_glucose_level", "bmi")]
numerical_vars$stroke = stroke_data$stroke
cor_matrix = cor(numerical_vars, use = "complete.obs")
print(cor_matrix)
```

```
##           age avg_glucose_level      bmi      stroke
## age           1.0000000      0.2383228 0.33519923 0.24523876
## avg_glucose_level 0.2383228      1.0000000 0.17267144 0.13199119
## bmi              0.3351992      0.1726714 1.00000000 0.04332213
## stroke           0.2452388      0.1319912 0.04332213 1.00000000
```

```
library(caret)
```

```
## Loading required package: lattice
```

```
##
```

```
## Attaching package: 'caret'
```

```
## The following object is masked from 'package:purrr':
```

```
##
```

```
## lift
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
```

```
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      cov, smooth, var
```

```
stroke_data = stroke_data %>%  
  select(-id, -age_group, -Residence_type, -gender) %>%  
  mutate(  
    ever_married = as.factor(ever_married),  
    work_type = as.factor(work_type),  
    smoking_status = as.factor(smoking_status),  
    hypertension = as.factor(hypertension),  
    heart_disease = as.factor(heart_disease),  
    stroke = as.factor(stroke)  
  )  
stroke_data = stroke_data %>%  
  mutate(stroke = factor(stroke, levels = c(1, 0), labels = c("YES", "NO")))
```

```
set.seed(123)  
trainIndex = createDataPartition(stroke_data$stroke, p = 0.7, list = FALSE)  
train_data = stroke_data[trainIndex, ]  
test_data = stroke_data[-trainIndex, ]  
prop.table(table(train_data$stroke))
```

```
##  
##      YES      NO  
## 0.04892368 0.95107632
```

```
prop.table(table(test_data$stroke))
```

```
##  
##      YES      NO  
## 0.04830287 0.95169713
```

Define Training Controls

We'll define two `trainControl` objects for 5-fold cross-validation: one without SMOTE and one with SMOTE for class balancing. We'll use these across all models and scenarios.

```
# Without SMOTE  
ctrl_base = trainControl(  
  method = "cv",  
  number = 5,
```

```

    classProbs = TRUE,
    summaryFunction = twoClassSummary,
  )

ctrl_smote = trainControl(
  method = "cv",
  number = 5,
  classProbs = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final",
  sampling = "smote"
)

```

For each model (Logistic Regression, LDA, QDA, KNN), we'll train under four scenarios: 1. **No SMOTE, No PCA** 2. **SMOTE, No PCA** 3. **No SMOTE, PCA** 4. **SMOTE, PCA**

Logistic Regression

```

# No SMOTE, No PCA
set.seed(123)
logit_no_smote_no_pca = train(
  stroke ~ .,
  data = train_data,
  method = "glm",
  family = "binomial",
  trControl = ctrl_base,
  metric = "ROC"
)
logit_pred_no_smote_no_pca_prob <- predict(logit_no_smote_no_pca, newdata = test_data, type = "prob")
logit_pred_no_smote_no_pca_class <- predict(logit_no_smote_no_pca, newdata = test_data, type = "raw")
roc_logit_no_smote_no_pca <- roc(test_data$stroke, logit_pred_no_smote_no_pca_prob[, "YES"])

## Setting levels: control = YES, case = NO

## Setting direction: controls > cases

acc_logit_no_smote_no_pca <- confusionMatrix(logit_pred_no_smote_no_pca_class, test_data$stroke)$overall

# SMOTE, No PCA
set.seed(123)
logit_smote_no_pca = train(
  stroke ~ .,
  data = train_data,
  method = "glm",
  family = "binomial",
  trControl = ctrl_smote,
  metric = "ROC"
)

## Loading required package: recipes

```

```

##
## Attaching package: 'recipes'

## The following object is masked from 'package:stringr':
##
##     fixed

## The following object is masked from 'package:stats':
##
##     step

logit_pred_smote_no_pca_prob <- predict(logit_smote_no_pca, newdata = test_data, type = "prob")
logit_pred_smote_no_pca_class <- predict(logit_smote_no_pca, newdata = test_data, type = "raw")
roc_logit_smote_no_pca <- roc(test_data$stroke, logit_pred_smote_no_pca_prob[, "YES"])

## Setting levels: control = YES, case = NO

## Setting direction: controls > cases

acc_logit_smote_no_pca <- confusionMatrix(logit_pred_smote_no_pca_class, test_data$stroke)$overall["Acc"]

# No SMOTE, PCA
set.seed(123)
logit_no_smote_pca = train(
  stroke ~ .,
  data = train_data,
  method = "glm",
  family = "binomial",
  trControl = ctrl_base,
  preProcess = c("center", "scale", "pca"), # PCA with centering and scaling
  metric = "ROC"
)

## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

logit_pred_no_smote_pca_prob <- predict(logit_no_smote_pca, newdata = test_data, type = "prob")
logit_pred_no_smote_pca_class <- predict(logit_no_smote_pca, newdata = test_data, type = "raw")
roc_logit_no_smote_pca <- roc(test_data$stroke, logit_pred_no_smote_pca_prob[, "YES"])

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

acc_logit_no_smote_pca <- confusionMatrix(logit_pred_no_smote_pca_class, test_data$stroke)$overall["Acc"]

# SMOTE, PCA
set.seed(123)
logit_smote_pca = train(
  stroke ~ .,
  data = train_data,

```

```

method = "glm",
family = "binomial",
trControl = ctrl_smote,
preProcess = c("center", "scale", "pca"),
metric = "ROC"
)

```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```

logit_pred_smote_pca_prob <- predict(logit_smote_pca, newdata = test_data, type = "prob")
roc_logit_smote_pca <- roc(test_data$stroke, logit_pred_smote_pca_prob[, "YES"])

```

```

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

```

```

logit_pred_smote_pca_class <- predict(logit_smote_pca, newdata = test_data, type = "raw")
acc_logit_smote_pca <- confusionMatrix(logit_pred_smote_pca_class, test_data$stroke)$overall["Accuracy"]

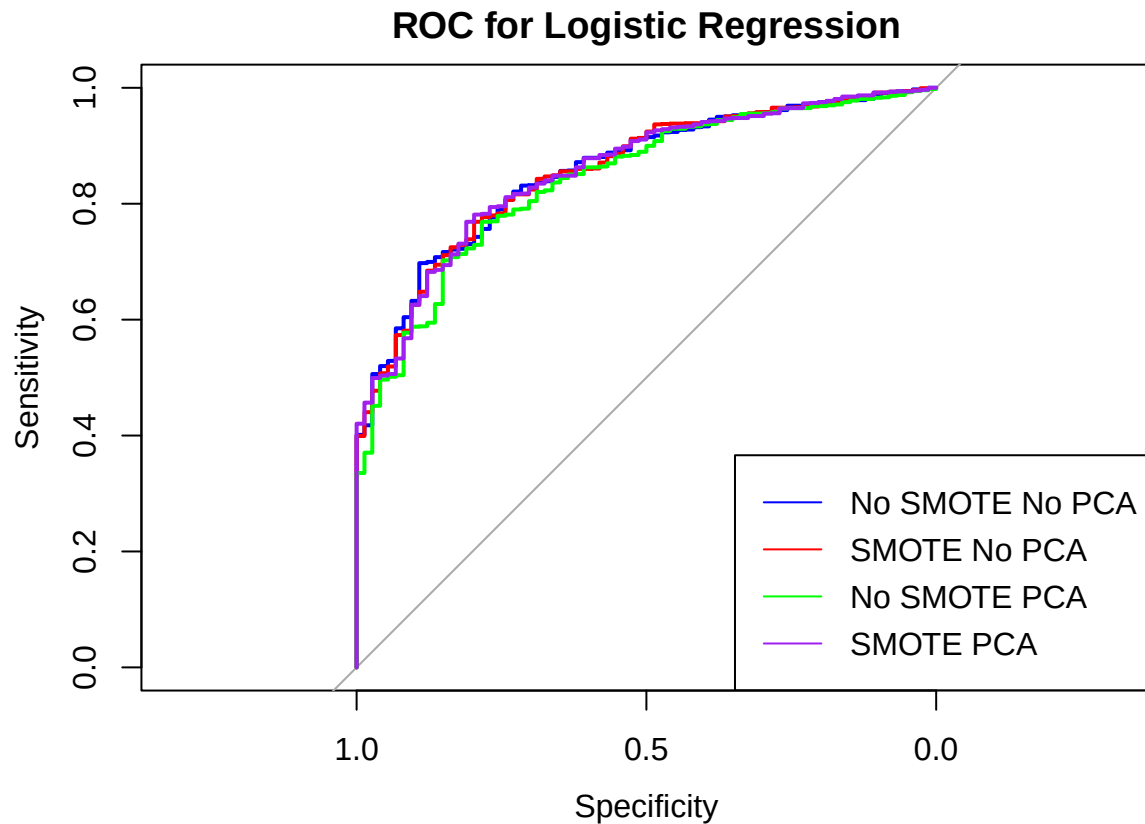
```

```
# Plot ROC curves
```

```

plot(roc_logit_no_smote_no_pca, col = "blue", main = "ROC for Logistic Regression")
lines(roc_logit_smote_no_pca, col = "red")
lines(roc_logit_no_smote_pca, col = "green")
lines(roc_logit_smote_pca, col = "purple")
legend("bottomright",
      legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE PCA"),
      col = c("blue", "red", "green", "purple"),
      lty = 1)

```



```

### Linear Discriminant Analysis (LDA)
# No SMOTE, No PCA
set.seed(123)
lda_no_smote_no_pca = train(
  stroke ~ .,
  data = train_data,
  method = "lda",
  trControl = ctrl_base,
  metric = "ROC"
)
lda_pred_no_smote_no_pca = predict(lda_no_smote_no_pca, newdata = test_data, type = "prob")
roc_lda_no_smote_no_pca = roc(test_data$stroke, lda_pred_no_smote_no_pca[, "YES"])

## Setting levels: control = YES, case = NO

## Setting direction: controls > cases

lda_pred_no_smote_no_pca_class <- predict(lda_no_smote_no_pca, newdata = test_data, type = "raw")
acc_lda_no_smote_no_pca <- confusionMatrix(lda_pred_no_smote_no_pca_class, test_data$stroke)$overall["A"]

# SMOTE, No PCA
set.seed(123)
lda_smote_no_pca = train(
  stroke ~ .,

```

```

data = train_data,
method = "lda",
trControl = ctrl_smote,
metric = "ROC"
)
lda_pred_smote_no_pca = predict(lda_smote_no_pca, newdata = test_data, type = "prob")
roc_lda_smote_no_pca = roc(test_data$stroke, lda_pred_smote_no_pca[, "YES"])

```

```

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

```

```

lda_pred_smote_no_pca_class <- predict(lda_smote_no_pca, newdata = test_data, type = "raw")
acc_lda_smote_no_pca <- confusionMatrix(lda_pred_smote_no_pca_class, test_data$stroke)$overall["Accuracy"]

```

```

# No SMOTE, PCA

```

```

set.seed(123)
lda_no_smote_pca = train(
  stroke ~ .,
  data = train_data,
  method = "lda",
  trControl = ctrl_base,
  preProcess = c("center", "scale", "pca"),
  metric = "ROC"
)
lda_pred_no_smote_pca = predict(lda_no_smote_pca, newdata = test_data, type = "prob")
roc_lda_no_smote_pca = roc(test_data$stroke, lda_pred_no_smote_pca[, "YES"])

```

```

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

```

```

lda_pred_no_smote_pca_class <- predict(lda_no_smote_pca, newdata = test_data, type = "raw")
acc_lda_no_smote_pca <- confusionMatrix(lda_pred_no_smote_pca_class, test_data$stroke)$overall["Accuracy"]

```

```

# SMOTE, PCA

```

```

set.seed(123)
lda_smote_pca = train(
  stroke ~ .,
  data = train_data,
  method = "lda",
  trControl = ctrl_smote,
  preProcess = c("center", "scale", "pca"),
  metric = "ROC"
)
lda_pred_smote_pca = predict(lda_smote_pca, newdata = test_data, type = "prob")
roc_lda_smote_pca = roc(test_data$stroke, lda_pred_smote_pca[, "YES"])

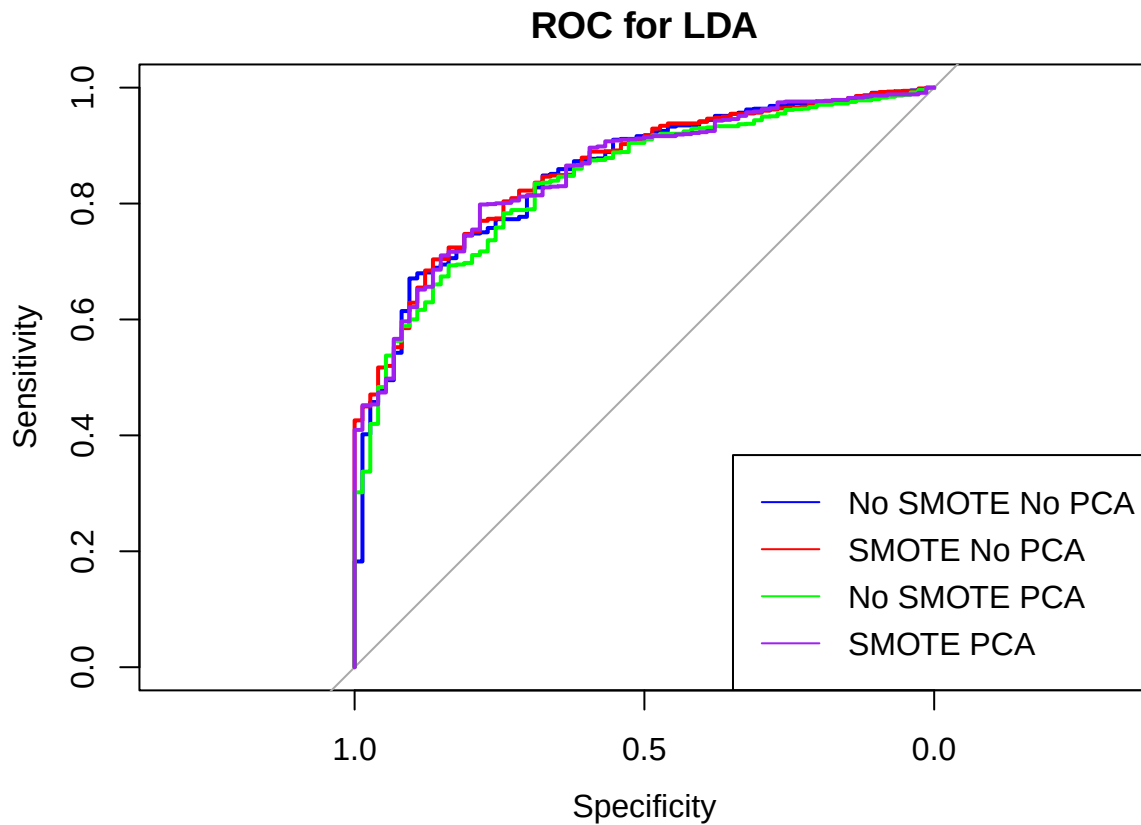
```

```

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

```

```
lda_pred_smote_pca_class <- predict(lda_smote_pca, newdata = test_data, type = "raw")
acc_lda_smote_pca <- confusionMatrix(lda_pred_smote_pca_class, test_data$stroke)$overall["Accuracy"]
# Plot ROC curves
plot(roc_lda_no_smote_no_pca, col = "blue", main = "ROC for LDA")
lines(roc_lda_smote_no_pca, col = "red")
lines(roc_lda_no_smote_pca, col = "green")
lines(roc_lda_smote_pca, col = "purple")
legend("bottomright",
      legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE PCA"),
      col = c("blue", "red", "green", "purple"),
      lty = 1)
```



```
### Quadratic Discriminant Analysis (QDA)
```

```
# No SMOTE, No PCA
set.seed(123)
qda_no_smote_no_pca <- tryCatch({
  train(
    stroke ~ .,
    data = train_data,
    method = "qda",
    trControl = ctrl_base,
    metric = "ROC"
  )
}, error = function(e) NULL)
```

```
## Warning: model fit failed for Fold1: parameter=none Error in qda.default(x, grouping, ...) : rank de
## Warning: model fit failed for Fold2: parameter=none Error in qda.default(x, grouping, ...) : rank de
## Warning: model fit failed for Fold3: parameter=none Error in qda.default(x, grouping, ...) : rank de
## Warning: model fit failed for Fold4: parameter=none Error in qda.default(x, grouping, ...) : rank de
## Warning: model fit failed for Fold5: parameter=none Error in qda.default(x, grouping, ...) : rank de

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
## Something is wrong; all the ROC metric values are missing:
```

```
##      ROC      Sens      Spec
## Min.   : NA   Min.   : NA   Min.   : NA
## 1st Qu.: NA   1st Qu.: NA   1st Qu.: NA
## Median : NA   Median : NA   Median : NA
## Mean   :NaN   Mean   :NaN   Mean   :NaN
## 3rd Qu.: NA   3rd Qu.: NA   3rd Qu.: NA
## Max.   : NA   Max.   : NA   Max.   : NA
## NA's   :1     NA's   :1     NA's   :1
```

```
qda_pred_no_smote_no_pca <- if (!is.null(qda_no_smote_no_pca)) {
  predict(qda_no_smote_no_pca, newdata = test_data, type = "prob")
} else {
  NULL
}

roc_qda_no_smote_no_pca <- if (!is.null(qda_pred_no_smote_no_pca)) {
  roc(test_data$stroke, qda_pred_no_smote_no_pca[, "YES"])
} else {
  NULL
}

qda_pred_no_smote_no_pca_class <- if (!is.null(qda_no_smote_no_pca)) {
  predict(qda_no_smote_no_pca, newdata = test_data, type = "raw")
} else {
  NULL
}

acc_qda_no_smote_no_pca <- if (!is.null(qda_pred_no_smote_no_pca_class)) {
  confusionMatrix(qda_pred_no_smote_no_pca_class, test_data$stroke)$overall["Accuracy"]
} else {
  NA
}

# SMOTE, No PCA
set.seed(123)
qda_smote_no_pca <- tryCatch({
  train(
    stroke ~ .,
    data = train_data,
    method = "qda",
    trControl = ctrl_smote,
```

```

    metric = "ROC"
  )
}, error = function(e) NULL)

```

```

## Warning: model fit failed for Fold1: parameter=none Error in qda.default(x, grouping, ...) : rank de
## Warning: model fit failed for Fold2: parameter=none Error in qda.default(x, grouping, ...) : rank de
## Warning: model fit failed for Fold3: parameter=none Error in qda.default(x, grouping, ...) : rank de
## Warning: model fit failed for Fold4: parameter=none Error in qda.default(x, grouping, ...) : rank de
## Warning: model fit failed for Fold5: parameter=none Error in qda.default(x, grouping, ...) : rank de

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

```

```

## Something is wrong; all the ROC metric values are missing:
##      ROC      Sens      Spec
## Min.   : NA   Min.   : NA   Min.   : NA
## 1st Qu.: NA   1st Qu.: NA   1st Qu.: NA
## Median : NA   Median : NA   Median : NA
## Mean   :NaN   Mean   :NaN   Mean   :NaN
## 3rd Qu.: NA   3rd Qu.: NA   3rd Qu.: NA
## Max.   : NA   Max.   : NA   Max.   : NA
## NA's   :1     NA's   :1     NA's   :1

```

```

qda_pred_smote_no_pca <- if (!is.null(qda_smote_no_pca)) {
  predict(qda_smote_no_pca, newdata = test_data, type = "prob")
} else {
  NULL
}

roc_qda_smote_no_pca <- if (!is.null(qda_pred_smote_no_pca)) {
  roc(test_data$stroke, qda_pred_smote_no_pca[, "YES"])
} else {
  NULL
}

qda_pred_smote_no_pca_class <- if (!is.null(qda_smote_no_pca)) {
  predict(qda_smote_no_pca, newdata = test_data, type = "raw")
} else {
  NULL
}

acc_qda_smote_no_pca <- if (!is.null(qda_pred_smote_no_pca_class)) {
  confusionMatrix(qda_pred_smote_no_pca_class, test_data$stroke)$overall["Accuracy"]
} else {
  NA
}

# No SMOTE, PCA
set.seed(123)
qda_no_smote_pca <- train(

```

```

stroke ~ .,
data = train_data,
method = "qda",
trControl = ctrl_base,
preProcess = c("center", "scale", "pca"),
metric = "ROC"
)
qda_pred_no_smote_pca <- predict(qda_no_smote_pca, newdata = test_data, type = "prob")
roc_qda_no_smote_pca <- roc(test_data$stroke, qda_pred_no_smote_pca[, "YES"])

```

```
## Setting levels: control = YES, case = NO
```

```
## Setting direction: controls > cases
```

```

qda_pred_no_smote_pca_class <- predict(qda_no_smote_pca, newdata = test_data, type = "raw")
acc_qda_no_smote_pca <- confusionMatrix(qda_pred_no_smote_pca_class, test_data$stroke)$overall["Accuracy"]

```

```
# SMOTE, PCA
```

```
set.seed(123)
```

```

qda_smote_pca <- train(
  stroke ~ .,
  data = train_data,
  method = "qda",
  trControl = ctrl_smote,
  preProcess = c("center", "scale", "pca"),
  metric = "ROC"
)

```

```

qda_pred_smote_pca <- predict(qda_smote_pca, newdata = test_data, type = "prob")
roc_qda_smote_pca <- roc(test_data$stroke, qda_pred_smote_pca[, "YES"])

```

```
## Setting levels: control = YES, case = NO
```

```
## Setting direction: controls > cases
```

```

qda_pred_smote_pca_class <- predict(qda_smote_pca, newdata = test_data, type = "raw")
acc_qda_smote_pca <- confusionMatrix(qda_pred_smote_pca_class, test_data$stroke)$overall["Accuracy"]

```

```
# Plot ROC curves (only for successful models)
```

```
plot(roc_qda_no_smote_pca, col = "green", main = "ROC for QDA")
```

```
lines(roc_qda_smote_pca, col = "purple")
```

```
if (!is.null(roc_qda_no_smote_no_pca)) lines(roc_qda_no_smote_no_pca, col = "blue")
```

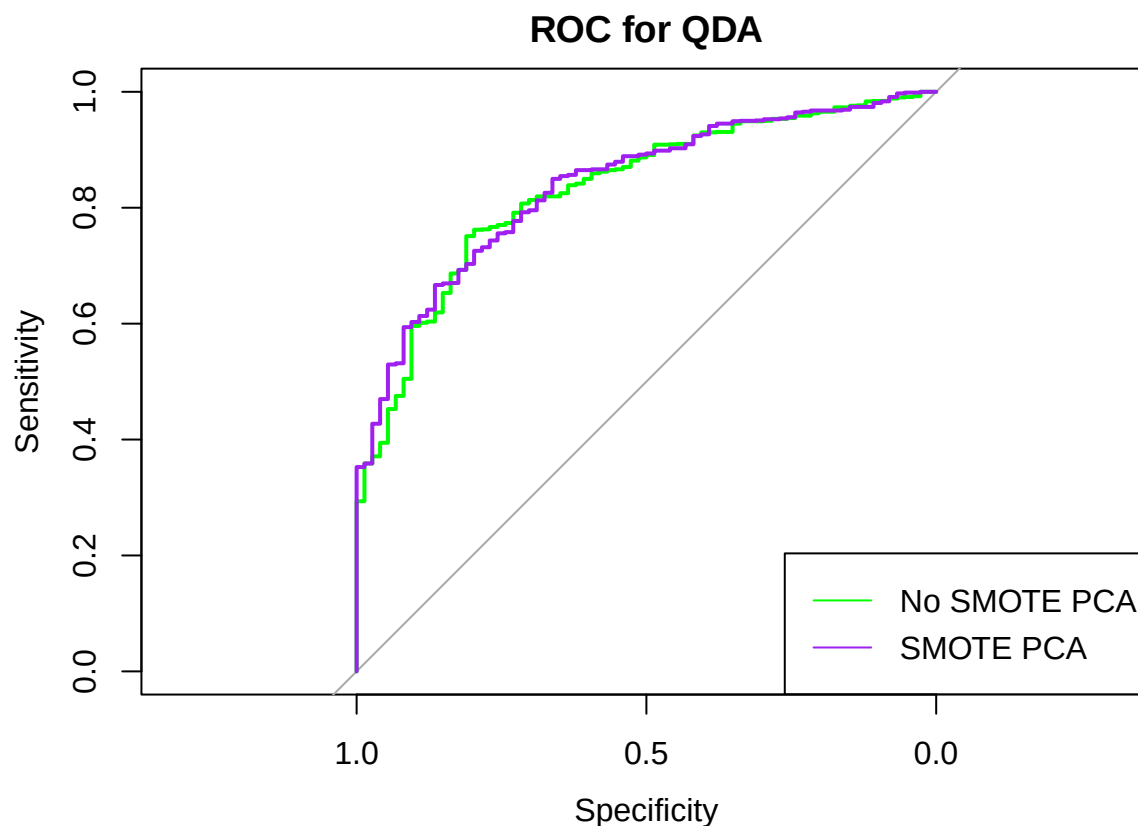
```
if (!is.null(roc_qda_smote_no_pca)) lines(roc_qda_smote_no_pca, col = "red")
```

```
legend("bottomright",
```

```
  legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE PCA")[c(!is.null(roc_qda_no_
```

```
  col = c("blue", "red", "green", "purple")[c(!is.null(roc_qda_no_smote_no_pca), !is.null(roc_qda_
```

```
  lty = 1)
```



K-Nearest Neighbors (KNN)

No SMOTE, No PCA

```
set.seed(123)
knn_no_smote_no_pca = train(
  stroke ~ .,
  data = train_data,
  method = "knn",
  trControl = ctrl_base,
  metric = "ROC"
)
knn_pred_no_smote_no_pca = predict(knn_no_smote_no_pca, newdata = test_data, type = "prob")
roc_knn_no_smote_no_pca = roc(test_data$stroke, knn_pred_no_smote_no_pca[, "YES"])
```

Setting levels: control = YES, case = NO

Setting direction: controls > cases

```
knn_pred_no_smote_no_pca_class <- predict(knn_no_smote_no_pca, newdata = test_data, type = "raw")
acc_knn_no_smote_no_pca <- confusionMatrix(knn_pred_no_smote_no_pca_class, test_data$stroke)$overall["A"]
```

SMOTE, No PCA

```
set.seed(123)
knn_smote_no_pca = train(
  stroke ~ .,
```

```

data = train_data,
method = "knn",
trControl = ctrl_smote,
metric = "ROC"
)
knn_pred_smote_no_pca = predict(knn_smote_no_pca, newdata = test_data, type = "prob")
roc_knn_smote_no_pca = roc(test_data$stroke, knn_pred_smote_no_pca[, "YES"])

```

```

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

```

```

knn_pred_smote_no_pca_class <- predict(knn_smote_no_pca, newdata = test_data, type = "raw")
acc_knn_smote_no_pca <- confusionMatrix(knn_pred_smote_no_pca_class, test_data$stroke)$overall["Accuracy"]

```

No SMOTE, PCA

```

set.seed(123)
knn_no_smote_pca = train(
  stroke ~ .,
  data = train_data,
  method = "knn",
  trControl = ctrl_base,
  preProcess = c("center", "scale", "pca"),
  metric = "ROC"
)
knn_pred_no_smote_pca = predict(knn_no_smote_pca, newdata = test_data, type = "prob")
roc_knn_no_smote_pca = roc(test_data$stroke, knn_pred_no_smote_pca[, "YES"])

```

```

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

```

```

knn_pred_no_smote_pca_class <- predict(knn_no_smote_pca, newdata = test_data, type = "raw")
acc_knn_no_smote_pca <- confusionMatrix(knn_pred_no_smote_pca_class, test_data$stroke)$overall["Accuracy"]

```

SMOTE, PCA

```

set.seed(123)
knn_smote_pca = train(
  stroke ~ .,
  data = train_data,
  method = "knn",
  trControl = ctrl_smote,
  preProcess = c("center", "scale", "pca"),
  metric = "ROC"
)
knn_pred_smote_pca = predict(knn_smote_pca, newdata = test_data, type = "prob")
roc_knn_smote_pca = roc(test_data$stroke, knn_pred_smote_pca[, "YES"])

```

```

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

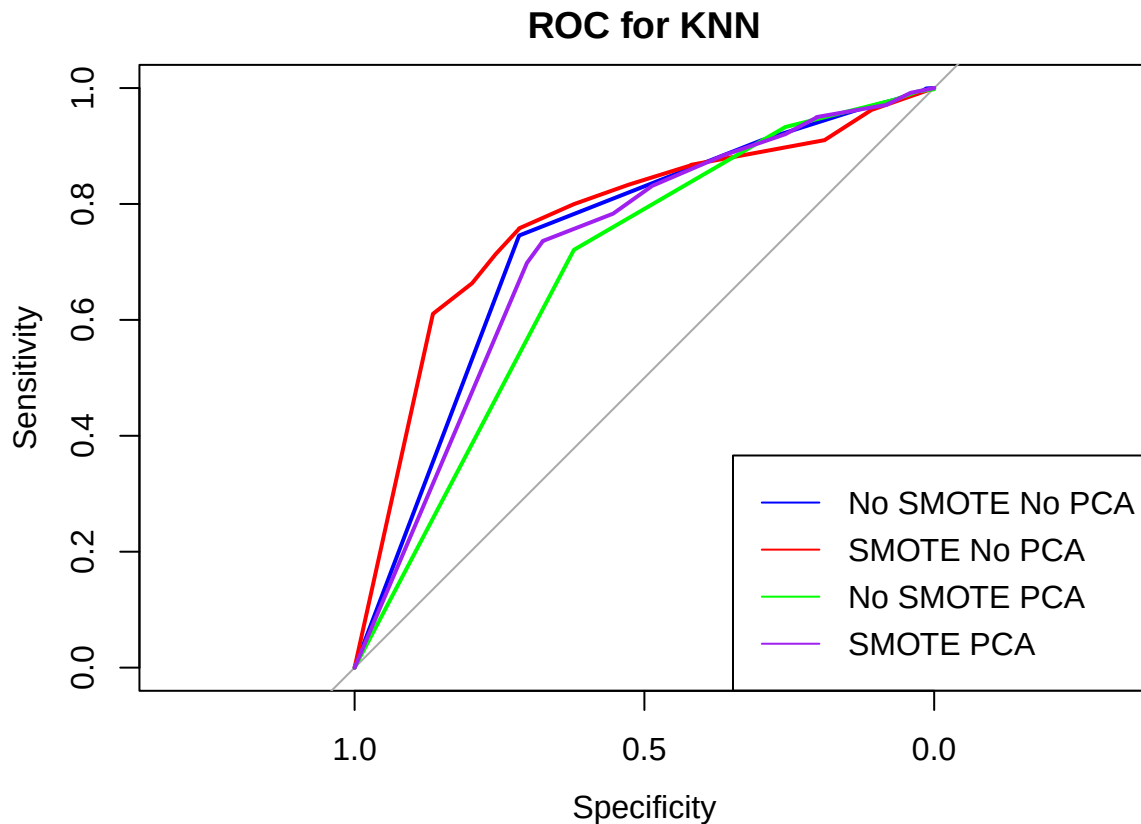
```

```

knn_pred_smote_pca_class <- predict(knn_smote_pca, newdata = test_data, type = "raw")
acc_knn_smote_pca <- confusionMatrix(knn_pred_smote_pca_class, test_data$stroke)$overall["Accuracy"]

```

```
# Plot ROC curves
plot(roc_knn_no_smote_no_pca, col = "blue", main = "ROC for KNN")
lines(roc_knn_smote_no_pca, col = "red")
lines(roc_knn_no_smote_pca, col = "green")
lines(roc_knn_smote_pca, col = "purple")
legend("bottomright",
      legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE PCA"),
      col = c("blue", "red", "green", "purple"),
      lty = 1)
```



```
library(gbm)      # For boosting (gbm)
```

```
## Loaded gbm 2.2.2
```

```
## This version of gbm is no longer under development. Consider transitioning to gbm3, https://github.com
```

```
library(rpart)    # For decision trees
library(ipred)    # For bagging
library(tibble)   # For AUC data frame
library(dplyr)    # For data manipulation
library(knitr)    # For table formatting
```

```

#Decision Tree
# No SMOTE, No PCA
set.seed(123)
dt_no_smote_no_pca <- train(
  stroke ~ .,
  data = train_data,
  method = "rpart",
  trControl = ctrl_base,
  tuneLength = 5, # Tune complexity parameter
  metric = "ROC"
)
dt_pred_no_smote_no_pca_prob <- predict(dt_no_smote_no_pca, newdata = test_data, type = "prob")
dt_pred_no_smote_no_pca_class <- predict(dt_no_smote_no_pca, newdata = test_data, type = "raw")
roc_dt_no_smote_no_pca <- roc(test_data$stroke, dt_pred_no_smote_no_pca_prob[, "YES"])

## Setting levels: control = YES, case = NO

## Setting direction: controls > cases

acc_dt_no_smote_no_pca <- confusionMatrix(dt_pred_no_smote_no_pca_class, test_data$stroke)$overall["Accuracy"]

# SMOTE, No PCA
set.seed(123)
dt_smote_no_pca <- train(
  stroke ~ .,
  data = train_data,
  method = "rpart",
  trControl = ctrl_smote,
  tuneLength = 5,
  metric = "ROC"
)
dt_pred_smote_no_pca_prob <- predict(dt_smote_no_pca, newdata = test_data, type = "prob")
dt_pred_smote_no_pca_class <- predict(dt_smote_no_pca, newdata = test_data, type = "raw")
roc_dt_smote_no_pca <- roc(test_data$stroke, dt_pred_smote_no_pca_prob[, "YES"])

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

acc_dt_smote_no_pca <- confusionMatrix(dt_pred_smote_no_pca_class, test_data$stroke)$overall["Accuracy"]

# No SMOTE, PCA
set.seed(123)
dt_no_smote_pca <- train(
  stroke ~ .,
  data = train_data,
  method = "rpart",
  trControl = ctrl_base,
  preProcess = c("center", "scale", "pca"),
  tuneLength = 5,
  metric = "ROC"
)

```

```

)
dt_pred_no_smote_pca_prob <- predict(dt_no_smote_pca, newdata = test_data, type = "prob")
dt_pred_no_smote_pca_class <- predict(dt_no_smote_pca, newdata = test_data, type = "raw")
roc_dt_no_smote_pca <- roc(test_data$stroke, dt_pred_no_smote_pca_prob[, "YES"])

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

acc_dt_no_smote_pca <- confusionMatrix(dt_pred_no_smote_pca_class, test_data$stroke)$overall["Accuracy"]

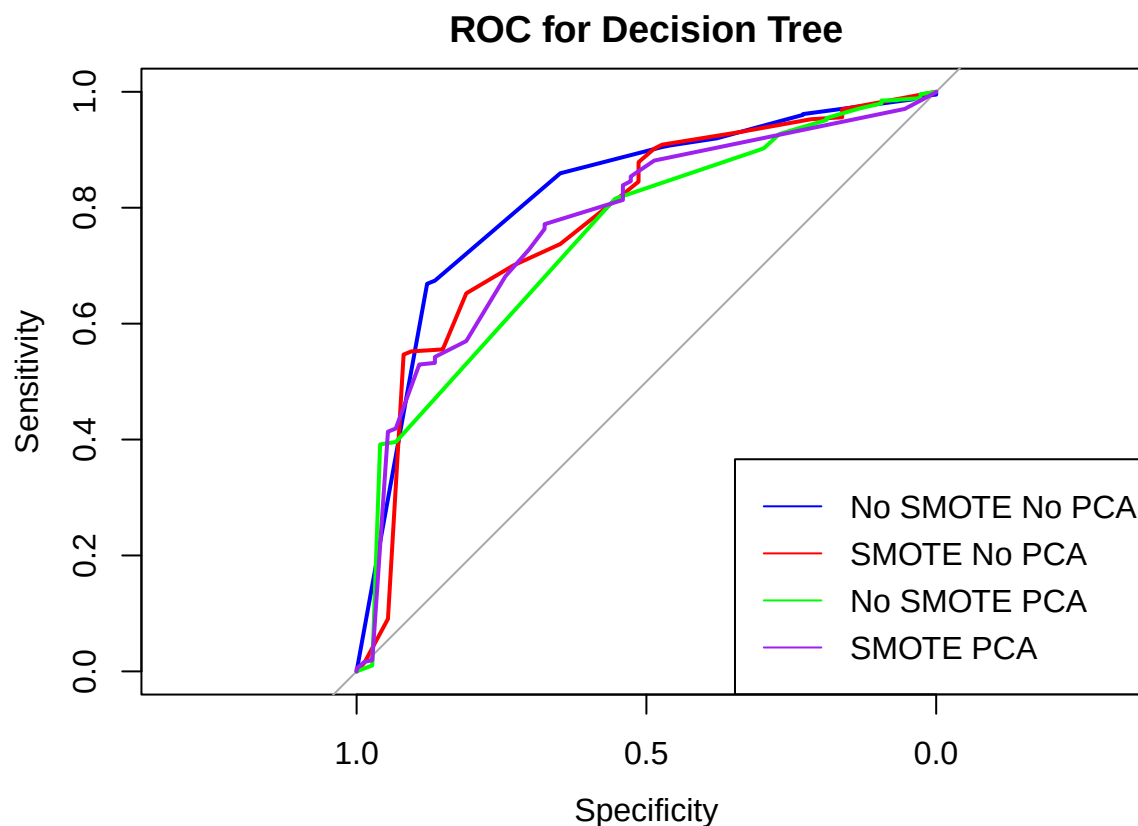
# SMOTE, PCA
set.seed(123)
dt_smote_pca <- train(
  stroke ~ .,
  data = train_data,
  method = "rpart",
  trControl = ctrl_smote,
  preProcess = c("center", "scale", "pca"),
  tuneLength = 5,
  metric = "ROC"
)
dt_pred_smote_pca_prob <- predict(dt_smote_pca, newdata = test_data, type = "prob")
dt_pred_smote_pca_class <- predict(dt_smote_pca, newdata = test_data, type = "raw")
roc_dt_smote_pca <- roc(test_data$stroke, dt_pred_smote_pca_prob[, "YES"])

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

acc_dt_smote_pca <- confusionMatrix(dt_pred_smote_pca_class, test_data$stroke)$overall["Accuracy"]

# Plot ROC curves
plot(roc_dt_no_smote_no_pca, col = "blue", main = "ROC for Decision Tree")
lines(roc_dt_smote_no_pca, col = "red")
lines(roc_dt_no_smote_pca, col = "green")
lines(roc_dt_smote_pca, col = "purple")
legend("bottomright",
  legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE PCA"),
  col = c("blue", "red", "green", "purple"),
  lty = 1)

```



```
#Random Forest
# No SMOTE, No PCA
set.seed(123)
rf_no_smote_no_pca <- train(
  stroke ~ .,
  data = train_data,
  method = "rf",
  trControl = ctrl_base,
  tuneLength = 3, # Tune mtry
  metric = "ROC"
)
rf_pred_no_smote_no_pca <- predict(rf_no_smote_no_pca, newdata = test_data, type = "prob")
roc_rf_no_smote_no_pca <- roc(test_data$stroke, rf_pred_no_smote_no_pca[, "YES"])

## Setting levels: control = YES, case = NO

## Setting direction: controls > cases

rf_pred_no_smote_no_pca_class <- predict(rf_no_smote_no_pca, newdata = test_data, type = "raw")
acc_rf_no_smote_no_pca <- confusionMatrix(rf_pred_no_smote_no_pca_class, test_data$stroke)$overall["Acc"]

# SMOTE, No PCA
set.seed(123)
rf_smote_no_pca <- train(
  stroke ~ .,
```

```

data = train_data,
method = "rf",
trControl = ctrl_smote,
tuneLength = 3,
metric = "ROC"
)
rf_pred_smote_no_pca <- predict(rf_smote_no_pca, newdata = test_data, type = "prob")
roc_rf_smote_no_pca <- roc(test_data$stroke, rf_pred_smote_no_pca[, "YES"])

```

```

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

```

```

rf_pred_smote_no_pca_class <- predict(rf_smote_no_pca, newdata = test_data, type = "raw")
acc_rf_smote_no_pca <- confusionMatrix(rf_pred_smote_no_pca_class, test_data$stroke)$overall["Accuracy"]

```

```

# No SMOTE, PCA

```

```

set.seed(123)
rf_no_smote_pca <- train(
  stroke ~ .,
  data = train_data,
  method = "rf",
  trControl = ctrl_base,
  preProcess = c("center", "scale", "pca"),
  tuneLength = 3,
  metric = "ROC"
)

```

```

## Warning in randomForest.default(x, y, mtry = param$mtry, ...): invalid mtry:
## reset to within valid range
## Warning in randomForest.default(x, y, mtry = param$mtry, ...): invalid mtry:
## reset to within valid range
## Warning in randomForest.default(x, y, mtry = param$mtry, ...): invalid mtry:
## reset to within valid range
## Warning in randomForest.default(x, y, mtry = param$mtry, ...): invalid mtry:
## reset to within valid range
## Warning in randomForest.default(x, y, mtry = param$mtry, ...): invalid mtry:
## reset to within valid range

```

```

rf_pred_no_smote_pca <- predict(rf_no_smote_pca, newdata = test_data, type = "prob")
roc_rf_no_smote_pca <- roc(test_data$stroke, rf_pred_no_smote_pca[, "YES"])

```

```

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

```

```

rf_pred_no_smote_pca_class <- predict(rf_no_smote_pca, newdata = test_data, type = "raw")
acc_rf_no_smote_pca <- confusionMatrix(rf_pred_no_smote_pca_class, test_data$stroke)$overall["Accuracy"]

```

```

# SMOTE, PCA

```

```

set.seed(123)
rf_smote_pca <- train(
  stroke ~ .,

```

```

data = train_data,
method = "rf",
trControl = ctrl_smote,
preProcess = c("center", "scale", "pca"),
tuneLength = 3,
metric = "ROC"
)

```

```

## Warning in randomForest.default(x, y, mtry = param$mtry, ...): invalid mtry:
## reset to within valid range
## Warning in randomForest.default(x, y, mtry = param$mtry, ...): invalid mtry:
## reset to within valid range
## Warning in randomForest.default(x, y, mtry = param$mtry, ...): invalid mtry:
## reset to within valid range
## Warning in randomForest.default(x, y, mtry = param$mtry, ...): invalid mtry:
## reset to within valid range
## Warning in randomForest.default(x, y, mtry = param$mtry, ...): invalid mtry:
## reset to within valid range

```

```

rf_pred_smote_pca <- predict(rf_smote_pca, newdata = test_data, type = "prob")
roc_rf_smote_pca <- roc(test_data$stroke, rf_pred_smote_pca[, "YES"])

```

```

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

```

```

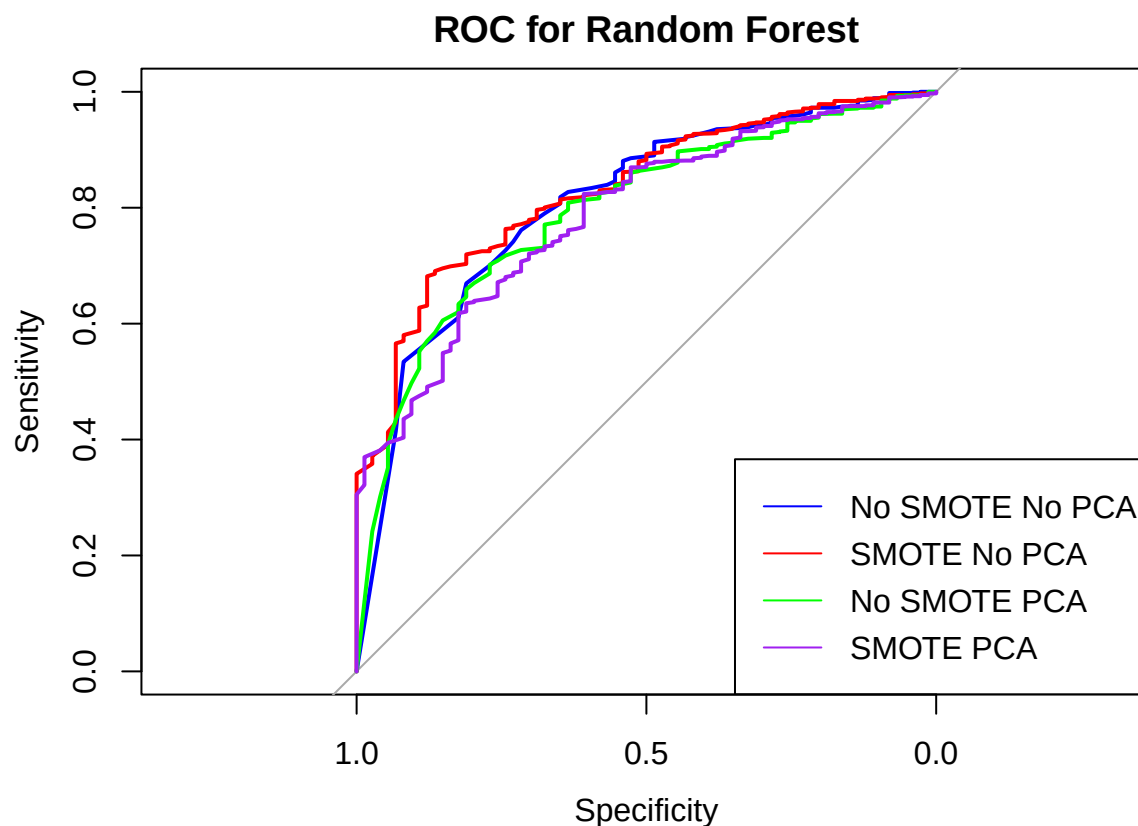
rf_pred_smote_pca_class <- predict(rf_smote_pca, newdata = test_data, type = "raw")
acc_rf_smote_pca <- confusionMatrix(rf_pred_smote_pca_class, test_data$stroke)$overall["Accuracy"]

```

```

# Plot ROC curves
plot(roc_rf_no_smote_no_pca, col = "blue", main = "ROC for Random Forest")
lines(roc_rf_smote_no_pca, col = "red")
lines(roc_rf_no_smote_pca, col = "green")
lines(roc_rf_smote_pca, col = "purple")
legend("bottomright",
      legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE PCA"),
      col = c("blue", "red", "green", "purple"),
      lty = 1)

```



```
#Bagging
# No SMOTE, No PCA
set.seed(123)
bag_no_smote_no_pca <- train(
  stroke ~ .,
  data = train_data,
  method = "treebag",
  trControl = ctrl_base,
  metric = "ROC"
)
bag_pred_no_smote_no_pca <- predict(bag_no_smote_no_pca, newdata = test_data, type = "prob")
roc_bag_no_smote_no_pca <- roc(test_data$stroke, bag_pred_no_smote_no_pca[, "YES"])
```

```
## Setting levels: control = YES, case = NO
```

```
## Setting direction: controls > cases
```

```
bag_pred_no_smote_no_pca_class <- predict(bag_no_smote_no_pca, newdata = test_data, type = "raw")
acc_bag_no_smote_no_pca <- confusionMatrix(bag_pred_no_smote_no_pca_class, test_data$stroke)$overall["A"]

# SMOTE, No PCA
set.seed(123)
bag_smote_no_pca <- train(
  stroke ~ .,
  data = train_data,
```

```

method = "treebag",
trControl = ctrl_smote,
metric = "ROC"
)
bag_pred_smote_no_pca <- predict(bag_smote_no_pca, newdata = test_data, type = "prob")
roc_bag_smote_no_pca <- roc(test_data$stroke, bag_pred_smote_no_pca[, "YES"])

```

```

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

```

```

bag_pred_smote_no_pca_class <- predict(bag_smote_no_pca, newdata = test_data, type = "raw")
acc_bag_smote_no_pca <- confusionMatrix(bag_pred_smote_no_pca_class, test_data$stroke)$overall["Accuracy"]

# No SMOTE, PCA
set.seed(123)
bag_no_smote_pca <- train(
  stroke ~ .,
  data = train_data,
  method = "treebag",
  trControl = ctrl_base,
  preProcess = c("center", "scale", "pca"),
  metric = "ROC"
)
bag_pred_no_smote_pca <- predict(bag_no_smote_pca, newdata = test_data, type = "prob")
roc_bag_no_smote_pca <- roc(test_data$stroke, bag_pred_no_smote_pca[, "YES"])

```

```

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

```

```

bag_pred_no_smote_pca_class <- predict(bag_no_smote_pca, newdata = test_data, type = "raw")
acc_bag_no_smote_pca <- confusionMatrix(bag_pred_no_smote_pca_class, test_data$stroke)$overall["Accuracy"]

# SMOTE, PCA
set.seed(123)
bag_smote_pca <- train(
  stroke ~ .,
  data = train_data,
  method = "treebag",
  trControl = ctrl_smote,
  preProcess = c("center", "scale", "pca"),
  metric = "ROC"
)
bag_pred_smote_pca <- predict(bag_smote_pca, newdata = test_data, type = "prob")
roc_bag_smote_pca <- roc(test_data$stroke, bag_pred_smote_pca[, "YES"])

```

```

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

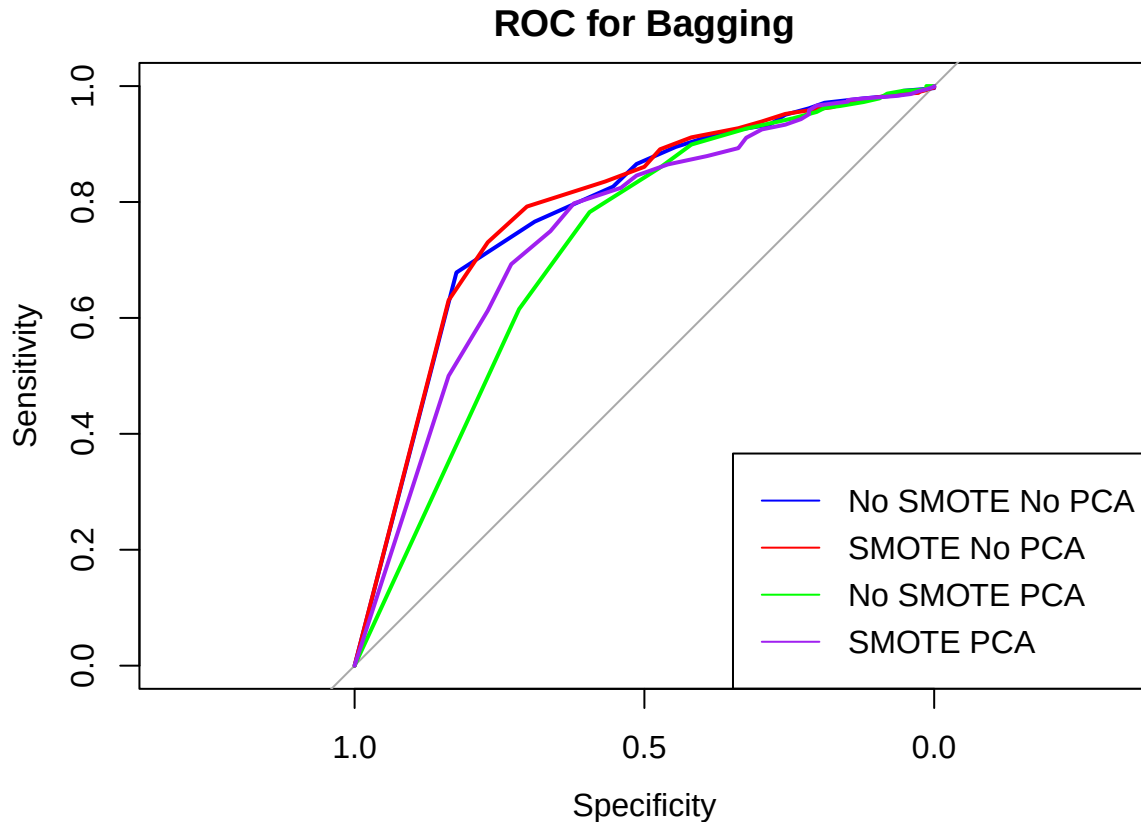
```

```

bag_pred_smote_pca_class <- predict(bag_smote_pca, newdata = test_data, type = "raw")
acc_bag_smote_pca <- confusionMatrix(bag_pred_smote_pca_class, test_data$stroke)$overall["Accuracy"]

```

```
# Plot ROC curves
plot(roc_bag_no_smote_no_pca, col = "blue", main = "ROC for Bagging")
lines(roc_bag_smote_no_pca, col = "red")
lines(roc_bag_no_smote_pca, col = "green")
lines(roc_bag_smote_pca, col = "purple")
legend("bottomright",
      legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE PCA"),
      col = c("blue", "red", "green", "purple"),
      lty = 1)
```



```
#Boosting
# No SMOTE, No PCA
set.seed(123)
boost_no_smote_no_pca <- train(
  stroke ~ .,
  data = train_data,
  method = "gbm",
  trControl = ctrl_base,
  tuneLength = 3, # Tune n.trees, interaction.depth, etc.
  metric = "ROC",
  verbose = FALSE
)
boost_pred_no_smote_no_pca <- predict(boost_no_smote_no_pca, newdata = test_data, type = "prob")
roc_boost_no_smote_no_pca <- roc(test_data$stroke, boost_pred_no_smote_no_pca[, "YES"])
```

```
## Setting levels: control = YES, case = NO
```

```
## Setting direction: controls > cases
```

```
boost_pred_no_smote_no_pca_class <- predict(boost_no_smote_no_pca, newdata = test_data, type = "raw")
acc_boost_no_smote_no_pca <- confusionMatrix(boost_pred_no_smote_no_pca_class, test_data$stroke)$overall
```

```
# SMOTE, No PCA
```

```
set.seed(123)
```

```
boost_smote_no_pca <- train(
  stroke ~ .,
  data = train_data,
  method = "gbm",
  trControl = ctrl_smote,
  tuneLength = 3,
  metric = "ROC",
  verbose = FALSE
)
```

```
boost_pred_smote_no_pca <- predict(boost_smote_no_pca, newdata = test_data, type = "prob")
```

```
roc_boost_smote_no_pca <- roc(test_data$stroke, boost_pred_smote_no_pca[, "YES"])
```

```
## Setting levels: control = YES, case = NO
```

```
## Setting direction: controls > cases
```

```
boost_pred_smote_no_pca_class <- predict(boost_no_smote_no_pca, newdata = test_data, type = "raw")
```

```
acc_boost_smote_no_pca <- confusionMatrix(boost_pred_smote_no_pca_class, test_data$stroke)$overall["Acc"]
```

```
# No SMOTE, PCA
```

```
set.seed(123)
```

```
boost_no_smote_pca <- train(
  stroke ~ .,
  data = train_data,
  method = "gbm",
  trControl = ctrl_base,
  preProcess = c("center", "scale", "pca"),
  tuneLength = 3,
  metric = "ROC",
  verbose = FALSE
)
```

```
boost_pred_no_smote_pca <- predict(boost_no_smote_pca, newdata = test_data, type = "prob")
```

```
roc_boost_no_smote_pca <- roc(test_data$stroke, boost_pred_no_smote_pca[, "YES"])
```

```
## Setting levels: control = YES, case = NO
```

```
## Setting direction: controls > cases
```

```
boost_pred_no_smote_pca_class <- predict(boost_no_smote_pca, newdata = test_data, type = "raw")
```

```
acc_boost_no_smote_pca <- confusionMatrix(boost_pred_no_smote_pca_class, test_data$stroke)$overall["Acc"]
```

```
# SMOTE, PCA
```

```
set.seed(123)
```

```
boost_smote_pca <- train(
  stroke ~ .,
```

```

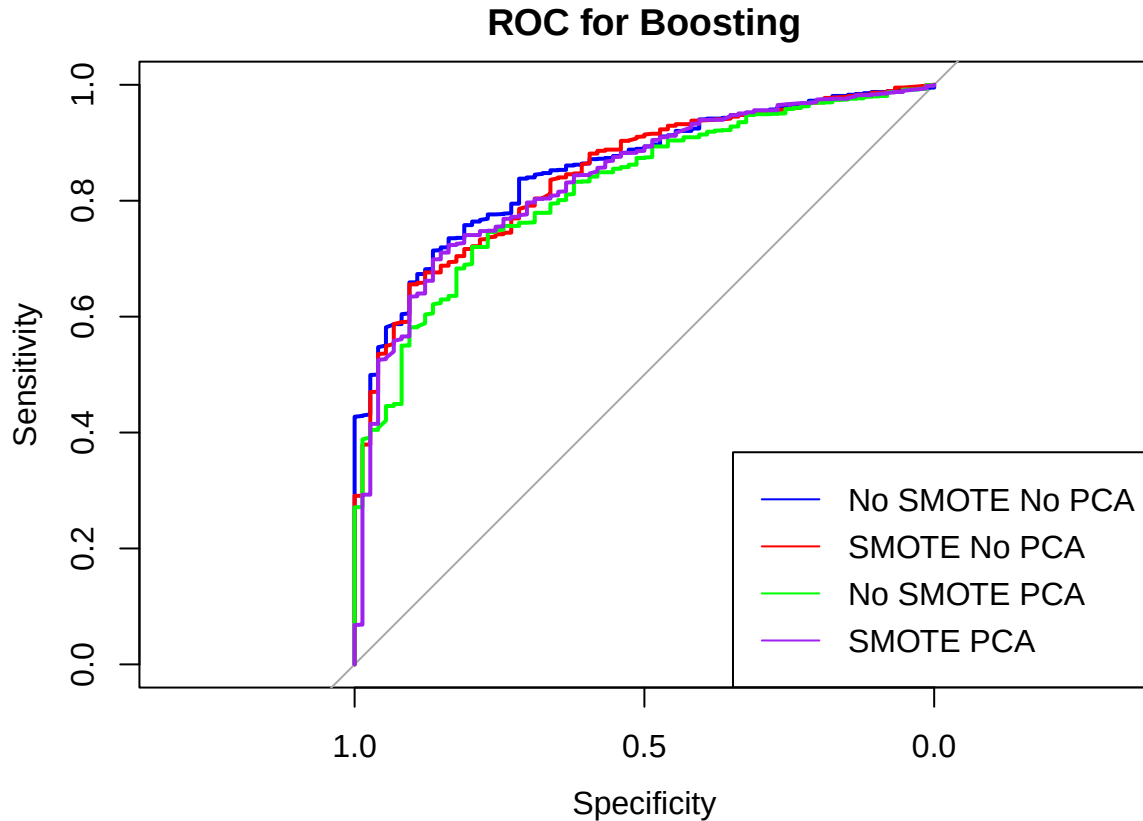
data = train_data,
method = "gbm",
trControl = ctrl_smote,
preProcess = c("center", "scale", "pca"),
tuneLength = 3,
metric = "ROC",
verbose = FALSE
)
boost_pred_smote_pca <- predict(boost_smote_pca, newdata = test_data, type = "prob")
roc_boost_smote_pca <- roc(test_data$stroke, boost_pred_smote_pca[, "YES"])

## Setting levels: control = YES, case = NO
## Setting direction: controls > cases

boost_pred_smote_pca_class <- predict(boost_smote_pca, newdata = test_data, type = "raw")
acc_boost_smote_pca <- confusionMatrix(boost_pred_smote_pca_class, test_data$stroke)$overall["Accuracy"]

# Plot ROC curves
plot(roc_boost_no_smote_no_pca, col = "blue", main = "ROC for Boosting")
lines(roc_boost_smote_no_pca, col = "red")
lines(roc_boost_no_smote_pca, col = "green")
lines(roc_boost_smote_pca, col = "purple")
legend("bottomright",
      legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE PCA"),
      col = c("blue", "red", "green", "purple"),
      lty = 1)

```



```

sens_logit_no_smote_no_pca <- confusionMatrix(logit_pred_no_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_logit_smote_no_pca <- confusionMatrix(logit_pred_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_logit_no_smote_pca <- confusionMatrix(logit_pred_no_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_logit_smote_pca <- confusionMatrix(logit_pred_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_lda_no_smote_no_pca <- confusionMatrix(lda_pred_no_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_lda_smote_no_pca <- confusionMatrix(lda_pred_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_lda_no_smote_pca <- confusionMatrix(lda_pred_no_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_lda_smote_pca <- confusionMatrix(lda_pred_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_lda_no_smote_no_pca <- confusionMatrix(lda_pred_no_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_lda_smote_no_pca <- confusionMatrix(lda_pred_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_lda_no_smote_pca <- confusionMatrix(lda_pred_no_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_lda_smote_pca <- confusionMatrix(lda_pred_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_qda_no_smote_no_pca <- if (!is.null(qda_pred_no_smote_no_pca_class)) {
  confusionMatrix(qda_pred_no_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
} else {
  NA
}
sens_qda_smote_no_pca <- if (!is.null(qda_pred_smote_no_pca_class)) {
  confusionMatrix(qda_pred_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
} else {
  NA
}
sens_qda_no_smote_pca <- confusionMatrix(qda_pred_no_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_qda_smote_pca <- confusionMatrix(qda_pred_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_dt_no_smote_no_pca <- confusionMatrix(dt_pred_no_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_dt_smote_no_pca <- confusionMatrix(dt_pred_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]

```

```
sens_dt_no_smote_pca <- confusionMatrix(dt_pred_no_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_dt_smote_pca <- confusionMatrix(dt_pred_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_rf_no_smote_no_pca <- confusionMatrix(rf_pred_no_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_rf_smote_no_pca <- confusionMatrix(rf_pred_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_rf_no_smote_pca <- confusionMatrix(rf_pred_no_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_rf_smote_pca <- confusionMatrix(rf_pred_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_bag_no_smote_no_pca <- confusionMatrix(bag_pred_no_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_bag_smote_no_pca <- confusionMatrix(bag_pred_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_bag_no_smote_pca <- confusionMatrix(bag_pred_no_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_bag_smote_pca <- confusionMatrix(bag_pred_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_boost_no_smote_no_pca <- confusionMatrix(boost_pred_no_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_boost_smote_no_pca <- confusionMatrix(boost_pred_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_boost_no_smote_pca <- confusionMatrix(boost_pred_no_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_boost_smote_pca <- confusionMatrix(boost_pred_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_knn_no_smote_no_pca <- confusionMatrix(knn_pred_no_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_knn_smote_no_pca <- confusionMatrix(knn_pred_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_knn_no_smote_pca <- confusionMatrix(knn_pred_no_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
sens_knn_smote_pca <- confusionMatrix(knn_pred_smote_pca_class, test_data$stroke)$byClass["Sensitivity"]
```

```

spec_bag_no_smote_pca <- confusionMatrix(bag_pred_no_smote_pca_class, test_data$stroke)$byClass["Specificity"]
spec_bag_smote_pca <- confusionMatrix(bag_pred_smote_pca_class, test_data$stroke)$byClass["Specificity"]
spec_boost_no_smote_no_pca <- confusionMatrix(boost_pred_no_smote_no_pca_class, test_data$stroke)$byClass["Specificity"]
spec_boost_smote_no_pca <- confusionMatrix(boost_pred_smote_no_pca_class, test_data$stroke)$byClass["Specificity"]
spec_boost_no_smote_pca <- confusionMatrix(boost_pred_no_smote_pca_class, test_data$stroke)$byClass["Specificity"]
spec_boost_smote_pca <- confusionMatrix(boost_pred_smote_pca_class, test_data$stroke)$byClass["Specificity"]
spec_knn_no_smote_no_pca <- confusionMatrix(knn_pred_no_smote_no_pca_class, test_data$stroke)$byClass["Specificity"]
spec_knn_smote_no_pca <- confusionMatrix(knn_pred_smote_no_pca_class, test_data$stroke)$byClass["Specificity"]
spec_knn_no_smote_pca <- confusionMatrix(knn_pred_no_smote_pca_class, test_data$stroke)$byClass["Specificity"]
spec_knn_smote_pca <- confusionMatrix(knn_pred_smote_pca_class, test_data$stroke)$byClass["Specificity"]

```

```

results_df <- data.frame(
  Model = rep(c("Logistic Regression", "LDA", "QDA", "Decision Tree",
               "Random Forest", "Bagging", "Boosting", "KNN"), each = 4),
  Scenario = rep(c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE PCA"), 8),
  AUC = c(
    auc(roc_logit_no_smote_no_pca), auc(roc_logit_smote_no_pca), auc(roc_logit_no_smote_pca), auc(roc_logit_smote_pca),
    auc(roc_lda_no_smote_no_pca), auc(roc_lda_smote_no_pca), auc(roc_lda_no_smote_pca), auc(roc_lda_smote_pca),
    if (!is.null(roc_qda_no_smote_no_pca)) auc(roc_qda_no_smote_no_pca) else NA,
    if (!is.null(roc_qda_smote_no_pca)) auc(roc_qda_smote_no_pca) else NA,
    auc(roc_qda_no_smote_pca), auc(roc_qda_smote_pca),
    auc(roc_dt_no_smote_no_pca), auc(roc_dt_smote_no_pca), auc(roc_dt_no_smote_pca), auc(roc_dt_smote_pca),
    auc(roc_rf_no_smote_no_pca), auc(roc_rf_smote_no_pca), auc(roc_rf_no_smote_pca), auc(roc_rf_smote_pca),
    auc(roc_bag_no_smote_no_pca), auc(roc_bag_smote_no_pca), auc(roc_bag_no_smote_pca), auc(roc_bag_smote_pca),
    auc(roc_boost_no_smote_no_pca), auc(roc_boost_smote_no_pca), auc(roc_boost_no_smote_pca), auc(roc_boost_smote_pca),
    auc(roc_knn_no_smote_no_pca), auc(roc_knn_smote_no_pca), auc(roc_knn_no_smote_pca), auc(roc_knn_smote_pca)
  ),
  Accuracy = c(
    acc_logit_no_smote_no_pca, acc_logit_smote_no_pca, acc_logit_no_smote_pca, acc_logit_smote_pca,
    acc_lda_no_smote_no_pca, acc_lda_smote_no_pca, acc_lda_no_smote_pca, acc_lda_smote_pca,
    acc_qda_no_smote_no_pca, acc_qda_smote_no_pca, acc_qda_no_smote_pca, acc_qda_smote_pca,
    acc_dt_no_smote_no_pca, acc_dt_smote_no_pca, acc_dt_no_smote_pca, acc_dt_smote_pca,
    acc_rf_no_smote_no_pca, acc_rf_smote_no_pca, acc_rf_no_smote_pca, acc_rf_smote_pca,
    acc_bag_no_smote_no_pca, acc_bag_smote_no_pca, acc_bag_no_smote_pca, acc_bag_smote_pca,
    acc_boost_no_smote_no_pca, acc_boost_smote_no_pca, acc_boost_no_smote_pca, acc_boost_smote_pca,
    acc_knn_no_smote_no_pca, acc_knn_smote_no_pca, acc_knn_no_smote_pca, acc_knn_smote_pca
  ),
  Sensitivity = c(
    sens_logit_no_smote_no_pca, sens_logit_smote_no_pca, sens_logit_no_smote_pca, sens_logit_smote_pca,
    sens_lda_no_smote_no_pca, sens_lda_smote_no_pca, sens_lda_no_smote_pca, sens_lda_smote_pca,
    sens_qda_no_smote_no_pca, sens_qda_smote_no_pca, sens_qda_no_smote_pca, sens_qda_smote_pca,
    sens_dt_no_smote_no_pca, sens_dt_smote_no_pca, sens_dt_no_smote_pca, sens_dt_smote_pca,
    sens_rf_no_smote_no_pca, sens_rf_smote_no_pca, sens_rf_no_smote_pca, sens_rf_smote_pca,
    sens_bag_no_smote_no_pca, sens_bag_smote_no_pca, sens_bag_no_smote_pca, sens_bag_smote_pca,
    sens_boost_no_smote_no_pca, sens_boost_smote_no_pca, sens_boost_no_smote_pca, sens_boost_smote_pca,
    sens_knn_no_smote_no_pca, sens_knn_smote_no_pca, sens_knn_no_smote_pca, sens_knn_smote_pca
  ),
  Specificity = c(
    spec_logit_no_smote_no_pca, spec_logit_smote_no_pca, spec_logit_no_smote_pca, spec_logit_smote_pca,
    spec_lda_no_smote_no_pca, spec_lda_smote_no_pca, spec_lda_no_smote_pca, spec_lda_smote_pca,
    spec_qda_no_smote_no_pca, spec_qda_smote_no_pca, spec_qda_no_smote_pca, spec_qda_smote_pca,
    spec_dt_no_smote_no_pca, spec_dt_smote_no_pca, spec_dt_no_smote_pca, spec_dt_smote_pca,
    spec_rf_no_smote_no_pca, spec_rf_smote_no_pca, spec_rf_no_smote_pca, spec_rf_smote_pca,

```

```

spec_bag_no_smote_no_pca, spec_bag_smote_no_pca, spec_bag_no_smote_pca, spec_bag_smote_pca,
spec_boost_no_smote_no_pca, spec_boost_smote_no_pca, spec_boost_no_smote_pca, spec_boost_smote_pca,
spec_knn_no_smote_no_pca, spec_knn_smote_no_pca, spec_knn_no_smote_pca, spec_knn_smote_pca
)
) %>%
  filter(!is.na(AUC) & !is.na(Accuracy) & !is.na(Specificity))

kable(results_df, caption = "AUC, Accuracy,Sensitivity, and Specificity for Each Model and Scenario (NA

```

Table 1: AUC, Accuracy,Sensitivity, and Specificity for Each Model and Scenario (NA Removed)

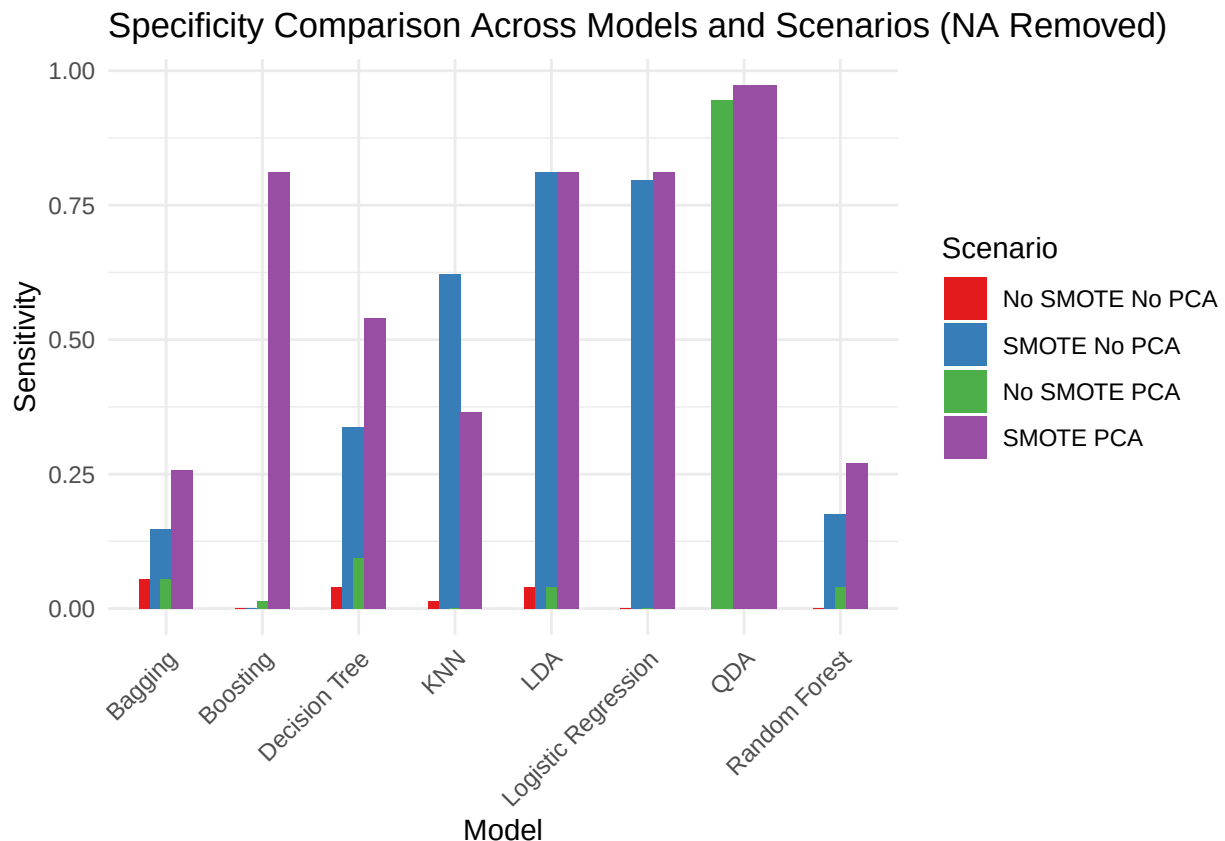
Model	Scenario	AUC	Accuracy	Sensitivity	Specificity
Logistic Regression	No SMOTE No PCA	0.8570	0.9517	0.0000	1.0000
Logistic Regression	SMOTE No PCA	0.8558	0.7487	0.7973	0.7462
Logistic Regression	No SMOTE PCA	0.8411	0.9517	0.0000	1.0000
Logistic Regression	SMOTE PCA	0.8561	0.7448	0.8108	0.7414
LDA	No SMOTE No PCA	0.8505	0.9478	0.0405	0.9938
LDA	SMOTE No PCA	0.8566	0.7324	0.8108	0.7284
LDA	No SMOTE PCA	0.8389	0.9445	0.0405	0.9904
LDA	SMOTE PCA	0.8533	0.7265	0.8108	0.7222
QDA	No SMOTE PCA	0.8314	0.4563	0.9459	0.4314
QDA	SMOTE PCA	0.8380	0.4458	0.9730	0.4191
Decision Tree	No SMOTE No PCA	0.8211	0.9413	0.0405	0.9870
Decision Tree	SMOTE No PCA	0.7791	0.9021	0.3378	0.9307
Decision Tree	No SMOTE PCA	0.7493	0.9419	0.0946	0.9849
Decision Tree	SMOTE PCA	0.7738	0.8003	0.5405	0.8134
Random Forest	No SMOTE No PCA	0.8084	0.9517	0.0000	1.0000
Random Forest	SMOTE No PCA	0.8313	0.9452	0.1757	0.9842
Random Forest	No SMOTE PCA	0.7943	0.9478	0.0405	0.9938
Random Forest	SMOTE PCA	0.7899	0.9178	0.2703	0.9506
Bagging	No SMOTE No PCA	0.7836	0.9452	0.0541	0.9904
Bagging	SMOTE No PCA	0.7890	0.9328	0.1486	0.9726
Bagging	No SMOTE PCA	0.7189	0.9465	0.0541	0.9918
Bagging	SMOTE PCA	0.7506	0.9008	0.2568	0.9335
Boosting	No SMOTE No PCA	0.8557	0.9510	0.0000	0.9993
Boosting	SMOTE No PCA	0.8456	0.9510	0.0000	0.9993
Boosting	No SMOTE PCA	0.8204	0.9497	0.0135	0.9973
Boosting	SMOTE PCA	0.8370	0.7409	0.8108	0.7373
KNN	No SMOTE No PCA	0.7369	0.9517	0.0135	0.9993
KNN	SMOTE No PCA	0.7710	0.7911	0.6216	0.7997
KNN	No SMOTE PCA	0.6860	0.9497	0.0000	0.9979
KNN	SMOTE PCA	0.7194	0.8584	0.3649	0.8834

```

results_df$Scenario <- factor(results_df$Scenario,
                              levels = c("No SMOTE No PCA", "SMOTE No PCA",
                                           "No SMOTE PCA", "SMOTE PCA"))

ggplot(results_df, aes(x = Model, y = Sensitivity, fill = Scenario)) +
  geom_bar(stat = "identity", position = position_dodge(width = 0.45)) +
  labs(title = "Specificity Comparison Across Models and Scenarios (NA Removed)",
       x = "Model",
       y = "Sensitivity") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  scale_fill_brewer(palette = "Set1")

```

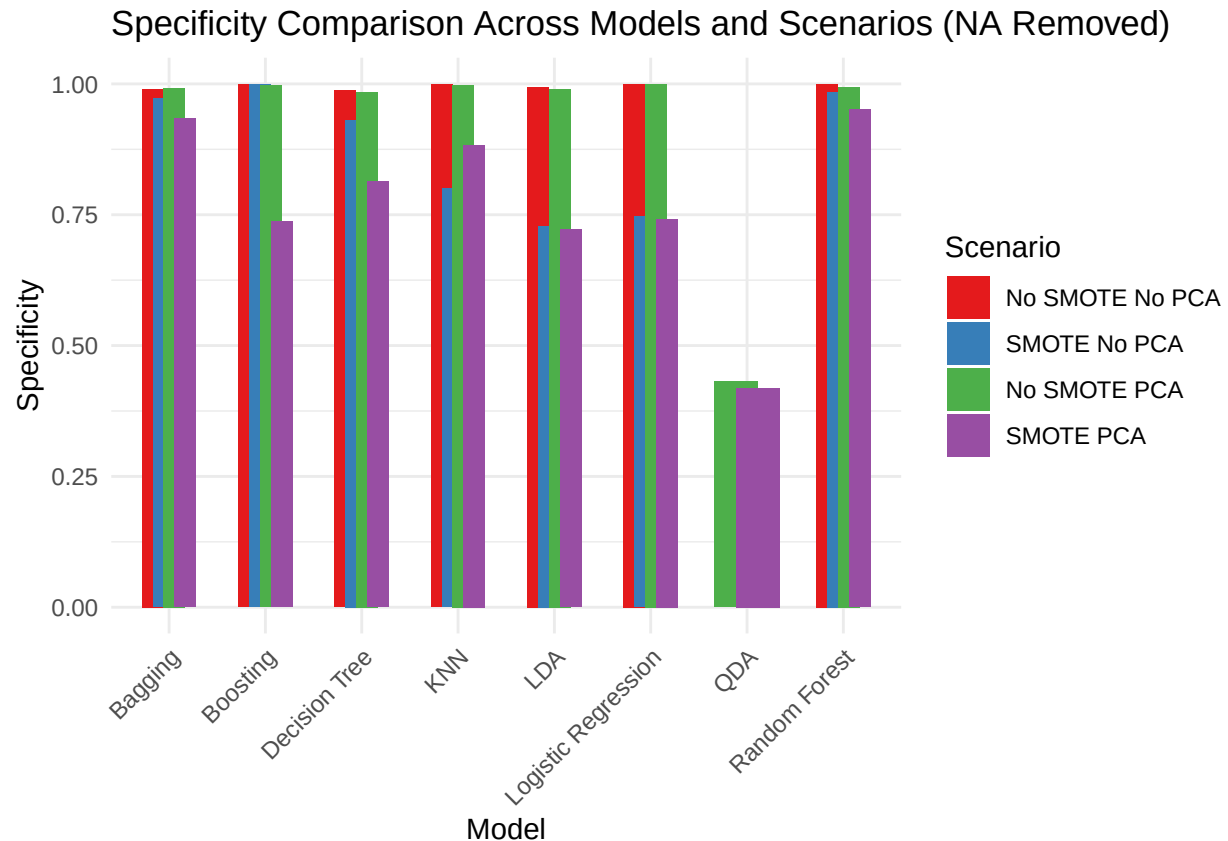


```

results_df$Scenario <- factor(results_df$Scenario,
                              levels = c("No SMOTE No PCA", "SMOTE No PCA",
                                           "No SMOTE PCA", "SMOTE PCA"))

ggplot(results_df, aes(x = Model, y = Specificity, fill = Scenario)) +
  geom_bar(stat = "identity", position = position_dodge(width = 0.45)) +
  labs(title = "Specificity Comparison Across Models and Scenarios (NA Removed)",
       x = "Model",
       y = "Specificity") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  scale_fill_brewer(palette = "Set1")

```

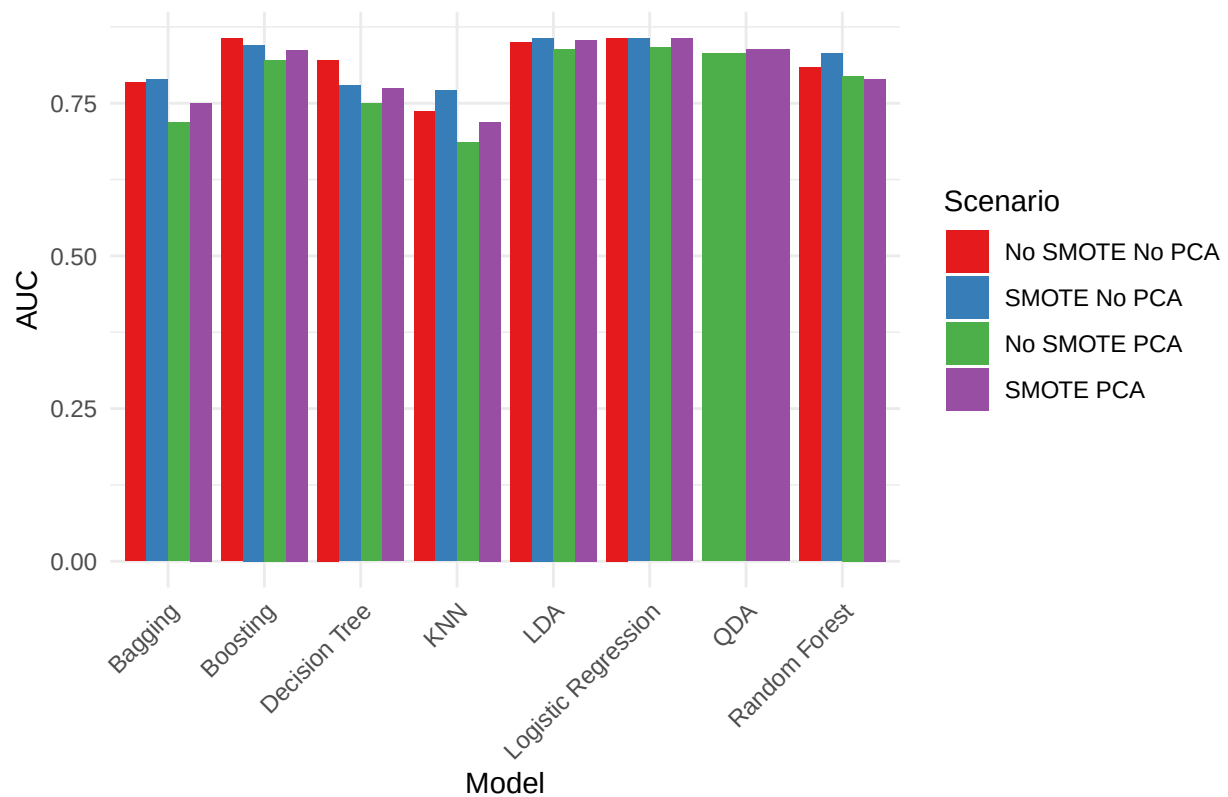


Because the QDA model fails to fit in every fold due to the rank deficiency issue, no predictions are produced. Without predictions, the performance metrics (like ROC) cannot be computed, leading to the “missing values” warning.

```
# Convert Scenario to factor for consistent ordering
results_df$Scenario <- factor(results_df$Scenario,
                              levels = c("No SMOTE No PCA", "SMOTE No PCA",
                                           "No SMOTE PCA", "SMOTE PCA"))

# Create grouped bar plot
ggplot(results_df, aes(x = Model, y = AUC, fill = Scenario)) +
  geom_bar(stat = "identity", position = position_dodge(width = 0.9)) +
  labs(title = "AUC Comparison Across Models and Scenarios (NA Removed)",
       x = "Model",
       y = "AUC") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  scale_fill_brewer(palette = "Set1")
```

AUC Comparison Across Models and Scenarios (NA Removed)



```
# Accuracy Plot
ggplot(results_df, aes(x = Model, y = Accuracy, fill = Scenario)) +
  geom_bar(stat = "identity", position = position_dodge(width = 0.45)) +
  labs(title = "Accuracy Comparison Across Models and Scenarios (NA Removed)",
       x = "Model",
       y = "Accuracy") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  scale_fill_brewer(palette = "Set1")
```

